國立臺灣大學電機資訊學院生醫電子與資訊學研究所

碩士論文(初稿)

Graduate Institute of Biomedical Electronics and Bioinformatics

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis (Draft)

基於基因體基礎模型之總體基因體短讀序列分類學分類

Token-level Genomic Foundation Model Fine-tuning for Metagenomic Taxonomic Classification

楊明儒

Ming-Ju Yang

指導教授: 劉子毓 博士

Advisor: Tzu-Yu Liu, Ph.D.

中華民國 115 年 7 月

July 2026

誌謝

基於基因體基礎模型之總體基因體短讀序列分類學分類

研究生: 楊明儒　　指導教授: 劉子毓 博士

國立臺灣大學　生醫電子與資訊學研究所

摘要

總體基因體學的分類學分類——即將每條定序短讀序列指派至正確的微生物分類單元——是微生物群落分析的核心計算挑戰。本論文提出一種基於基因體基礎模型的 Token 層級分類方法，利用預訓練的 Nucleotide Transformer v2（498M 參數）結合低秩適應（LoRA）微調與注意力池化分類頭，保留完整的 Token 層級序列資訊進行分類。透過系統性的資料規模實驗（500K 至 50M 條平衡讀序列），我們發現訓練資料量是決定分類準確度的主要因素：在屬（genus）層級 120 類分類任務中，資料量由 500K 增至 5M 可提升準確度 7.76 個百分點（55.29% 提升至 63.05%），進一步擴至 50M 可達 67.07%，而架構與訓練技巧的調整最多僅帶來 ±0.5 個百分點的變化。反向互補測試時增強（RC TTA）在所有設定下穩定提供 0.1–1.5 個百分點的準確度改善。此外，受控消融實驗證實預訓練 29 層骨幹網路比相同資料量下的單層隨機初始化模型高出 12.18 個百分點，驗證了基因體基礎模型預訓練表示的價值。本研究為基因體基礎模型在總體基因體學分類上的應用提供了實證基礎與可擴展的實驗框架。

關鍵字：

基因體基礎模型、總體基因體學、分類學分類、低秩適應、注意力池化

Token-level Genomic Foundation Model Fine-tuning for Metagenomic Taxonomic Classification

Student: Ming-Ju Yang Advisor: Tzu-Yu Liu, Ph.D.

Graduate Institute of Biomedical Electronics and Bioinformatics

National Taiwan University

ABSTRACT

Metagenomic taxonomic classification—assigning each sequencing read to its source organism in the microbial taxonomy—is a fundamental computational challenge in microbiome analysis. This thesis proposes a token-level classification approach based on genomic foundation models, leveraging the pre-trained Nucleotide Transformer v2 (498M parameters) with Low-Rank Adaptation (LoRA) fine-tuning and an attention-pooling classification head that preserves the full token-level sequence information.

Through systematic data scaling experiments spanning 500K to 50M balanced training reads, we demonstrate that training data volume is the dominant factor determining classification accuracy: a $10\times$ increase in training data yields a $+7.76$ percentage point improvement in genus-level accuracy (55.29% to 63.05% on 120 genera at 5M reads, scaling further to 67.07% at 50M reads), while architectural and training technique ablations produce at most ± 0.5 percentage points of variation. Reverse-complement test-time augmentation consistently improves accuracy by $+0.1$ to $+1.5$ percentage points across all experimental settings. Balanced subsampling across species reduces the number of unclassifiable taxa (F1=0 classes) from 9 to 0 at the 50M scale and improves macro F1 by $+5.08$ percentage points over the 5M baseline.

Comparison with MetaTransformer, a state-of-the-art Transformer-based metagenomic classifier achieving 98.3% genus recall on a large-scale balanced dataset (simulated from 2,505 HGR-UMGS genomes), reveals that the performance gap is primarily attributable to training data volume rather than model architecture. This work provides empirical evidence for the data scaling requirements of genomic foundation models in metagenomic classification and establishes a scalable experimental framework for further investigation.

Keywords:

Genomic Foundation Model, Metagenomics, Taxonomic Classification, LoRA, Attention Pooling

CONTENTS

LIST OF FIGURES

LIST OF TABLES

List of Symbols and Abbreviations

Abbreviations

AMP	Automatic Mixed Precision
ART	Artificial Read Technology (read simulator)
BPE	Byte Pair Encoding
DNA	Deoxyribonucleic Acid
ESM	Evolutionary Scale Modeling
FFN	Feed-Forward Network
GFM	Genomic Foundation Model
GPU	Graphics Processing Unit
HGR	Human Gastrointestinal Reference
LA	Logit Adjustment
LCA	Lowest Common Ancestor
LoRA	Low-Rank Adaptation
LR	Learning Rate
MLM	Masked Language Modeling
MLP	Multi-Layer Perceptron
NT	Nucleotide Transformer
PEFT	Parameter-Efficient Fine-Tuning
RC	Reverse Complement
RC TTA	Reverse-Complement Test-Time Augmentation
RoPE	Rotary Position Embedding
SLURM	Simple Linux Utility for Resource Management
TTA	Test-Time Augmentation
UMGS	Unified Human Gastrointestinal Genome collection
VRAM	Video Random Access Memory

Symbols

r	LoRA rank
α	LoRA scaling factor
K	Number of classes
T	Sequence length (number of tokens)
d	Hidden dimension of the backbone model
M	Number of learnable query tokens in attention pooling
W_Q, W_K, W_V	Query, key, and value projection matrices
A, B	LoRA low-rank decomposition matrices
τ	Temperature parameter for logit adjustment
π_k	Prior probability of class k
f_θ	Classifier parameterized by θ
$\bar{r}$	Reverse complement of read r

Chapter 1

Introduction

1.1 Background and Motivation

Metagenomics, the culture-independent study of microbial communities through direct sequencing of environmental DNA, has transformed our understanding of microbial ecology, human health, and environmental science [1]. Modern metagenomic sequencing platforms, particularly Illumina short-read technologies, generate hundreds of millions of reads per sample, each typically 100–300 base pairs (bp) in length. A fundamental computational challenge in metagenomic analysis is *taxonomic classification*: assigning each sequencing read to the correct taxon in the microbial taxonomy, such as genus or species [2].

Accurate taxonomic classification underpins virtually all downstream metagenomic analyses, including community composition profiling, pathogen detection, antibiotic resistance gene tracking, and comparative microbiome studies [3]. However, the classification task is inherently difficult due to several factors:

- **Short read length**: Individual reads of 150 bp carry limited phylogenetic signal, especially when distinguishing closely related taxa.
- **High taxonomic diversity**: A typical metagenomic sample may contain reads from hundreds to thousands of species spanning dozens of genera.
- **Class imbalance**: Community compositions follow highly skewed abundance distributions, with a few dominant taxa and many rare ones.

- **Sequence similarity**: Closely related taxa share substantial genomic homology, making fine-grained discrimination (e.g., species-level) particularly challenging.

Traditional approaches to metagenomic classification fall broadly into two categories. *Alignment-based methods* such as BLAST [4] and DIAMOND [5] align reads against reference databases, achieving high precision but at considerable computational cost. *k-mer-based methods* such as Kraken2 [6] and Centrifuge [7] index reference genomes using k-mer hash tables, enabling ultra-fast classification at the expense of sensitivity. More recently, deep learning approaches have emerged, including MetaTransformer [8], which applies a Transformer architecture to learned k-mer embeddings for metagenomic read classification.

Concurrently, the field of genomics has witnessed the development of *genomic foundation models* (GFMs)—large-scale pre-trained models that learn general-purpose representations of DNA sequences from billions of nucleotides [9]. Models such as the Nucleotide Transformer [9], DNABERT [10], DNABERT-2 [11], and Evo [12] have demonstrated strong performance across diverse genomic tasks, including promoter prediction, splice site identification, and variant effect prediction. These models offer a compelling alternative to task-specific feature engineering: by pre-training on vast genomic corpora, they capture rich sequence semantics that can be transferred to downstream tasks through fine-tuning.

Despite their promise, the application of GFMs to metagenomic taxonomic classification remains underexplored. A naïve approach—extracting mean-pooled embeddings from a frozen pre-trained model and training a linear classifier—discards the positional and local information encoded in individual token representations. This thesis investigates

whether a *token-level* approach, which preserves the full sequence of token embeddings and employs parameter-efficient fine-tuning, can substantially improve metagenomic taxonomic classification.

1.2 Problem Statement

Given a set of metagenomic short reads $\{r_1, r_2, \ldots, r_N\}$ generated by sequencing a microbial community, the taxonomic classification problem is to assign each read r_i to its correct taxon $y_i \in \{1, 2, \ldots, K\}$, where K is the number of classes at a given taxonomic level (e.g., genus or species).

This thesis formulates the problem as a supervised multi-class classification task. We assume access to a reference genome database from which simulated reads with known taxonomic labels can be generated for training. The challenge is to learn a classifier $f_\theta : \mathcal{R} \to \{1, \ldots, K\}$ that generalizes well from simulated training data to accurately classify novel reads.

The specific objectives of this thesis are:

1. **Token-level representation**: Investigate whether preserving the full sequence of token embeddings from a genomic foundation model, rather than using mean-pooled summaries, improves taxonomic classification accuracy.

2. **Parameter-efficient fine-tuning**: Evaluate the effectiveness of LoRA (Low-Rank Adaptation) [13] for adapting a 498M-parameter pre-trained model to the taxonomic classification task with minimal trainable parameters.

3. **Data scaling**: Quantify the relationship between training data volume and classifi-

cation performance, and determine the data requirements for achieving competitive accuracy.

4. **Comprehensive evaluation**: Assess performance across multiple metrics (micro accuracy, macro F1, top-k accuracy) and at both genus and species taxonomic levels.

1.3 Contributions

The main contributions of this thesis are as follows:

1. **A token-level GFM classification framework**: We propose a classification pipeline that feeds the complete token-level output of a pre-trained Nucleotide Transformer v2 [9] into a learnable attention-pooling classification head, fine-tuned with LoRA. This approach preserves rich positional and contextual information that is lost in mean-pooling approaches.

2. **Systematic data scaling analysis**: Through extensive experiments spanning 500K to 50M training reads, we demonstrate that training data volume is the dominant factor determining classification accuracy. Increasing training data from 500K to 5M balanced reads yields a $+7.8$ percentage point improvement in genus-level accuracy (55.3% $\rightarrow$ 63.1%), while architectural modifications alone yield at most ± 0.5 percentage points.

3. **Reverse-complement test-time augmentation**: We demonstrate that averaging logits from forward and reverse-complement passes at inference time consistently improves accuracy by $+0.1$ to $+1.5$ percentage points at no additional training cost, with larger gains for pre-trained models and smaller gains for randomly initialized architectures.

4. **Multi-level taxonomic evaluation**: We provide a comprehensive evaluation framework supporting genus-level (120 classes) and species-level (1,535 classes) classification, including oracle-genus evaluation, top-k genus routing, and per-class performance analysis.

5. **Controlled backbone ablation**: We design a controlled experiment comparing the pre-trained 29-layer NT-v2 backbone against a single-layer Transformer trained from scratch (MetaTransformer-style), using the same tokenizer and classification head. This isolates the contribution of pre-trained representations from model capacity and tokenization effects.

6. **Computational resource assessment**: We provide a detailed analysis of GPU memory, compute utilization, and training time on NVIDIA H100 hardware, offering practical guidance for scaling GFM-based classifiers to production metagenomic datasets.

1.4 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** reviews the background and related work, covering metagenomic classification methods, genomic foundation models, parameter-efficient fine-tuning, and attention mechanisms.

- **Chapter 3** presents the proposed methodology in detail, including the model architecture, training strategy, data pipeline, and evaluation framework.

- **Chapter 4** describes the experimental setup and presents comprehensive results,

including ablation studies, data scaling analysis, species-level evaluation, and computational resource assessment.

- **Chapter 5** summarizes the findings, discusses limitations, and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter provides the necessary background for understanding the methods proposed in this thesis. We review existing approaches to metagenomic taxonomic classification, introduce genomic foundation models, describe parameter-efficient fine-tuning techniques, and discuss attention-based sequence aggregation mechanisms.

2.1 Metagenomic Taxonomic Classification

Metagenomic taxonomic classification aims to assign each sequencing read from an environmental sample to its source organism's position in the phylogenetic tree. We review three major paradigms for this task.

2.1.1 Alignment-Based Methods

Alignment-based methods classify reads by aligning them to reference genome databases. BLAST [4] performs local sequence alignment using heuristic seed-and-extend strategies, achieving high sensitivity but at significant computational cost (hours to days for typical metagenomic datasets). DIAMOND [5] accelerates protein-level alignments through double indexing and optimized scoring, achieving up to 20,000× speedup over BLAST while maintaining comparable sensitivity. MegaBLAST [14] optimizes nucleotide-level alignment for closely related sequences.

The primary limitation of alignment-based methods is their computational cost, which scales poorly with the size of modern metagenomic datasets containing hundreds of mil-

lions of reads.

2.1.2 k-mer-Based Methods

k-mer-based methods represent sequences as sets or distributions of fixed-length subsequences (k-mers) and use hash-table lookups for rapid classification.

Kraken2 [6] constructs a database mapping each k-mer ($k = 35$) to its lowest common ancestor (LCA) in the taxonomic tree. Classification proceeds by querying each read's k-mers against this database and assigning the read to the taxon that covers the most k-mers. Kraken2 achieves classification speeds exceeding 10^6 reads per minute, making it one of the most widely adopted tools in metagenomic analysis.

Centrifuge [7] uses a compressed FM-index to perform rapid exact matching of reads against reference genomes, achieving lower memory requirements than Kraken2 while maintaining competitive accuracy.

While k-mer methods excel in speed, they rely on exact k-mer matches and thus struggle with sequences that diverge from the reference database. They also lack the ability to capture higher-order sequence patterns beyond individual k-mers.

2.1.3 Deep Learning Methods

Deep learning methods learn discriminative representations directly from sequence data, potentially capturing complex patterns that elude k-mer matching.

DeepMicrobes [15] applies a bidirectional LSTM with attention to one-hot encoded nucleotide sequences, achieving competitive performance on species-level classification

but requiring substantial training data and compute.

MetaTransformer [8] is the most directly relevant prior work to this thesis. It employs a custom Transformer architecture that operates on learned 12-mer embeddings. Key design choices include:

- **12-mer tokenization**: Input reads are segmented into non-overlapping 12-mers, each mapped to a learnable embedding vector. The choice of $k = 12$ balances vocabulary size ($4^{12} \approx 16.7$M possible 12-mers, though only $\sim$1.4M appear in training data) against sequence granularity.

- **Transformer encoder**: A Transformer encoder (the best-performing configuration uses $d_{\text{model}} = 64$ with a single encoder block for $k = 13$; the authors also evaluate up to 4 blocks and $d_{\text{model}} = 128$) processes the sequence of k-mer embeddings.

- **Mean pooling**: Token representations are mean-pooled into a fixed-length vector for classification.

- **Large-scale training**: The model is trained on balanced reads simulated from 2,505 HGR-UMGS genomes using the ART Illumina simulator with a coverage factor of 3 (the exact total number of reads is not reported in the paper; we estimate $\sim$614M based on 300K training steps $\times$ batch size 2,048). The best genus-level model achieves 98.9% precision and 98.3% recall on the validation set.

A key insight from MetaTransformer is the critical role of training data volume: the authors demonstrate that performance scales substantially with data size, and that balanced sampling across taxa is essential for achieving high macro-level performance.

2.2 Genomic Foundation Models

Genomic foundation models (GFMs) are large-scale neural networks pre-trained on vast corpora of DNA sequences using self-supervised learning objectives. Inspired by the success of language models such as BERT [16] and GPT [17], GFMs treat DNA sequences as a "language" of nucleotides and learn contextual representations that capture biological structure and function.

2.2.1 Nucleotide Transformer

The Nucleotide Transformer (NT) [9] family of models, developed by InstaDeep, are among the largest and most widely evaluated GFMs. This thesis uses the `nucleotide-transformer-v2-500m-multi-species` variant, which features:

- **Architecture**: An ESM-2-based [18] Transformer encoder with 29 layers, 1,024 hidden dimensions, and 16 attention heads.
- **Tokenization**: A 6-mer tokenizer that encodes DNA sequences into non-overlapping 6-nucleotide tokens. This produces a vocabulary of $4^6 = 4,096$ possible tokens plus special tokens.
- **Pre-training**: Masked language modeling (MLM) on 850 genomes from 174 species, spanning $\sim$3.2 billion nucleotides.
- **Total parameters**: 498M parameters.

The model produces contextual embeddings for each token in the input sequence, yielding a representation tensor of shape $[\text{batch}, \text{seq\_len}, 1024]$ that captures both local

sequence composition and long-range dependencies.

2.2.2 Other Genomic Foundation Models

DNABERT [10] was one of the first BERT-style models pre-trained on human genome sequences using overlapping k-mer tokenization ($k = 3, 4, 5, 6$). While effective for tasks such as promoter prediction, its relatively small scale (110M parameters) and human-genome-only pre-training limit its applicability to diverse metagenomic sequences.

DNABERT-2 [11] replaces k-mer tokenization with Byte Pair Encoding (BPE), eliminating the need to choose a fixed k and enabling more flexible sequence representation. It also adopts a multi-species pre-training strategy.

Evo [12] is a 7B-parameter model based on the StripedHyena architecture [19], capable of modeling DNA sequences up to 131K nucleotides. While powerful for long-range genomic modeling, its large size makes it challenging to deploy for metagenomic classification tasks involving hundreds of millions of short reads.

2.2.3 GFMs for Metagenomic Classification

Despite their success on single-genome tasks, GFMs have seen limited application to metagenomic classification. A naïve approach extracts mean-pooled embeddings from a frozen pre-trained model and trains a downstream classifier (e.g., MLP or SVM) on these fixed representations. While simple, this approach discards the token-level structure of the representation, losing positional information that may be discriminative for fine-grained taxonomic distinctions.

This thesis addresses this gap by proposing a token-level classification approach that preserves the full embedding sequence and fine-tunes the backbone with LoRA.

2.3 Parameter-Efficient Fine-Tuning

Fine-tuning the full parameter set of a 498M-parameter model requires substantial compute and memory, and risks catastrophic forgetting of the pre-trained representations. Parameter-efficient fine-tuning (PEFT) methods address this by adapting only a small subset of parameters while keeping the majority frozen.

2.3.1 Low-Rank Adaptation (LoRA)

LoRA [13] is based on the hypothesis that weight updates during fine-tuning lie in a low-rank subspace. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA parameterizes the update as:

$$W = W_0 + \Delta W = W_0 + BA \tag{2.1}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. During training, only A and B are updated while W_0 remains frozen. The forward pass becomes:

$$h = W_0 x + \frac{\alpha}{r} BAx \tag{2.2}$$

where α is a scaling hyperparameter that controls the magnitude of the adaptation.

Key advantages of LoRA include:

- **Parameter efficiency**: With $r = 16$ applied to query, key, and value projections across 29 Transformer layers, only 5.54M parameters (1.11% of total) are trainable.

- **No additional inference latency**: The adapted weights $W_0 + BA$ can be merged, adding no overhead at inference time.

- **Modular**: Different LoRA adapters can be trained for different tasks and swapped at deployment time.

2.3.2 Other PEFT Methods

Adapter layers [20] insert small trainable bottleneck modules between Transformer layers. While effective, they introduce additional inference latency due to the extra forward-pass computation.

Prefix tuning [21] prepends learnable "virtual tokens" to the input sequence, influencing the attention computation without modifying model weights. It is memory-efficient but can struggle with tasks requiring fine-grained output control.

We choose LoRA for this thesis due to its combination of strong performance, parameter efficiency, and the absence of additional inference latency.

2.4 Attention Mechanisms for Sequence Aggregation

Given a sequence of token embeddings $\{h_1, h_2, \ldots, h_T\} \in \mathbb{R}^{T \times d}$, a classification task requires aggregating these into a fixed-dimensional representation. The choice of aggregation mechanism directly impacts what information is preserved.

2.4.1 Mean Pooling

The simplest approach averages all token embeddings:

$$z = \frac{1}{T} \sum_{t=1}^{T} h_t \tag{2.3}$$

Mean pooling is computationally trivial but weights all tokens equally, potentially diluting the contribution of discriminative tokens with non-informative ones.

2.4.2 Attention Pooling

Attention pooling uses learnable query vectors to compute a weighted combination of token embeddings. Given M learnable query vectors $\{q_1, \ldots, q_M\} \in \mathbb{R}^{M \times d_q}$ and linear projections $W_K, W_V \in \mathbb{R}^{d \times d_q}$:

$$K = HW_K, \quad V = HW_V \tag{2.4}$$

$$\alpha_{m,t} = \frac{\exp(q_m \cdot k_t / \sqrt{d_q})}{\sum_{t'} \exp(q_m \cdot k_{t'} / \sqrt{d_q})} \tag{2.5}$$

$$z_m = \sum_{t=1}^{T} \alpha_{m,t} v_t \tag{2.6}$$

$$z = \text{Concat}(z_1, \ldots, z_M) \in \mathbb{R}^{M \cdot d_q} \tag{2.7}$$

This mechanism enables the model to learn which positions in the DNA sequence are most informative for classification, automatically focusing on discriminative k-mers while down-weighting uninformative regions.

2.4.3 Multi-Head Attention Pooling

Extending attention pooling with multiple heads allows the model to attend to different aspects of the sequence simultaneously. Each head learns a different query, capturing diverse patterns (e.g., one head may focus on conserved marker genes while another attends to variable regions). The concatenated outputs from all heads provide a rich, multi-faceted representation for downstream classification.

2.5 Data Augmentation for DNA Sequences

2.5.1 Reverse-Complement Symmetry in DNA

DNA is a double-stranded molecule: the two antiparallel strands are linked by Watson–Crick base pairing ($A \leftrightarrow T$, $C \leftrightarrow G$). Given a sequence $s = s_1 s_2 \cdots s_L$ on the forward (5′→3′) strand, its *reverse complement* $\bar{s} = \overline{s_L} \cdots \overline{s_2 s_1}$ on the complementary strand encodes identical biological information: both correspond to the same genomic locus and therefore the same organism.

In shotgun metagenomic sequencing, a read is equally likely to originate from either strand. Consequently, a biologically correct classifier must satisfy the *strand symmetry* property:

$$f(s) = f(\bar{s}) \quad \forall\, s \tag{2.8}$$

However, standard neural network architectures do not inherently enforce this constraint. Zhou et al. [22] systematically demonstrate that deep learning models for genomics frequently violate strand symmetry, producing inconsistent predictions for a sequence and its reverse complement. Ma [23] further show that even DNA language models fine-tuned with RC data augmentation still exhibit substantial prediction inconsistency.

2.5.2 Strategies for Exploiting RC Symmetry

Three complementary strategies have been proposed to address the strand symmetry problem:

1. **Random RC training augmentation**: During training, each read is replaced with its reverse complement with probability $p = 0.5$, exposing the model to both strand orientations. While this encourages—but does not guarantee—strand-invariant representations, it effectively doubles the training data diversity at zero storage cost.

2. **RC test-time augmentation (RC TTA)**: At inference, both the forward read s and its reverse complement $\bar{s}$ are processed independently, and their output logits are averaged before taking the argmax prediction:

$$\hat{y} = \arg\max_k \left[f_\theta(s) + f_\theta(\bar{s}) \right] \tag{2.9}$$

   This "post-hoc conjoined" approach [22] enforces strand symmetry at inference time without modifying the trained model. It can be understood as a two-view ensemble where both views observe the same biological entity from different strand orientations. By the standard ensemble argument, averaging two correlated but noisy estimates of the same ground-truth label reduces prediction variance and cancels orientation-dependent errors. Zhou et al. [22] find that this approach consistently performs well across diverse genomic tasks.

3. **RC consistency training**: A regularization loss (e.g., KL divergence) explicitly penalizes divergence between $f_\theta(s)$ and $f_\theta(\bar{s})$ during training, producing a single model that is intrinsically more strand-invariant without requiring double inference at test time [23]. An alternative is to design *RC-equivariant architectures* that hard-

code strand symmetry into the network structure [24], though this requires custom layers incompatible with pre-trained foundation model backbones.

In this thesis, we adopt strategies (1) and (2): RC augmentation during training and RC TTA during evaluation. We also experiment with RC consistency training (Section 4.4) but find that its interaction with other regularization techniques (e.g., logit adjustment) can be detrimental, and RC TTA alone provides a reliable $+1$–1.5 percentage point improvement across all experimental settings (Section 4.7).

2.6 Handling Class Imbalance

Metagenomic samples exhibit severe class imbalance: a few dominant taxa may constitute the majority of reads while rare taxa contribute only a handful. This imbalance can bias classifiers toward frequent classes, resulting in high micro accuracy but poor macro F1 due to frequent misclassification of rare taxa.

2.6.1 Loss-Level Approaches

Class-weighted cross-entropy scales the loss for each class inversely proportional to its frequency:

$$\mathcal{L} = -\sum_{k=1}^{K} w_k \, y_k \log \hat{y}_k, \quad w_k \propto 1/n_k \tag{2.10}$$

However, this approach can be unstable with mixed-precision training and large class counts [25].

Logit adjustment [26] modifies the output logits by a class-prior-dependent offset:

$$\tilde{f}_k(x) = f_k(x) + \tau \log \pi_k \tag{2.11}$$

where π_k is the prior probability of class k and τ is a temperature parameter controlling the strength of adjustment. This approach is theoretically motivated as producing Fisher-consistent estimators under class imbalance.

2.6.2 Sampling-Level Approaches

Balanced sampling constructs each training batch by sampling classes uniformly and then sampling instances within each class. This ensures that rare classes are seen as frequently as common ones during training.

Class-aware sampling (e.g., `WeightedRandomSampler`) assigns sampling weights inversely proportional to class frequency, achieving a similar effect to balanced batches within standard data loading pipelines.

2.7 Summary

This chapter has reviewed the key concepts underpinning our proposed approach: metagenomic classification methods from alignment-based to deep learning, genomic foundation models that provide rich pre-trained representations, parameter-efficient fine-tuning via LoRA, attention-based sequence aggregation, and strategies for data augmentation and class imbalance. In the next chapter, we describe how these components are integrated into a cohesive classification framework.

Chapter 3

Methodology

This chapter describes the proposed token-level genomic foundation model classification framework. We present the overall system architecture, detail each component, and describe the training strategy, data pipeline, and evaluation methodology.

3.1 System Overview

Figure 3.1 illustrates the end-to-end pipeline. Given a raw DNA read of 150 bp, the system performs the following steps:

1. **Tokenization**: The read is segmented into non-overlapping 6-mer tokens using the Nucleotide Transformer v2 tokenizer.

2. **Token-level encoding**: The token sequence is processed by the pre-trained Nucleotide Transformer v2 backbone (with LoRA adapters), producing contextual embeddings for each token.

3. **Attention pooling**: A multi-head attention pooling head aggregates the token embeddings into a fixed-dimensional classification vector.

4. **Classification**: A feed-forward network maps the pooled representation to class logits.

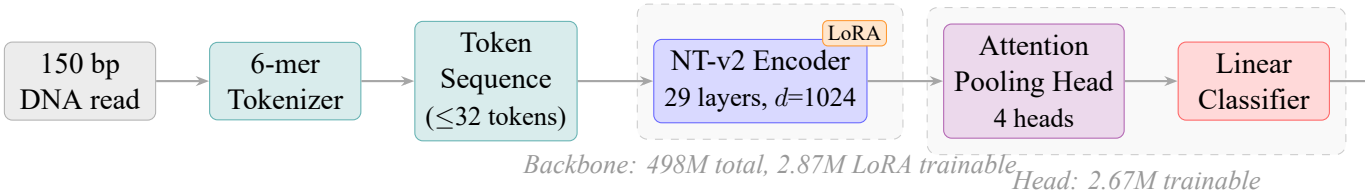

Figure 3.1: Overview of the proposed token-level GFM classification pipeline. A 150 bp DNA read is tokenized into non-overlapping 6-mers, encoded by the LoRA-adapted NT-v2 backbone (29 layers, d=1024), aggregated via multi-head attention pooling, and classified into one of K taxonomic classes (120 genera or 1,535 species). Orange badge indicates LoRA adapter modules; dashed boxes delineate the backbone and classification head parameter groups.

3.2 Backbone: Nucleotide Transformer v2

We employ the `nucleotide-transformer-v2-500m-multi-species` model as the backbone encoder. This model is based on the ESM-2 architecture [18], a protein language model architecture adapted for nucleotide sequences. The selection of NT-v2 over other genomic foundation models is motivated by four considerations:

1. **Empirical performance on metagenomic reads**: A direct comparison of frozen-backbone baselines (Section 4.2) shows that NT-v2 achieves 51.70% genus-level accuracy, outperforming the 7B-parameter Evo-2 model (46.95%) despite having 14× fewer parameters. This result provides direct empirical evidence that NT-v2's representations are better suited to metagenomic read classification.

2. **Multi-species pre-training**: NT-v2's `multi-species` variant is pre-trained on 850 genomes spanning 174 species [9], making its pre-training distribution the closest among available GFMs to the multi-organism nature of metagenomic data. By contrast, DNABERT [10] is pre-trained exclusively on the human genome, which is taxonomically far from gut microbial sequences.

3. **k-mer tokenization as a common ground with MetaTransformer**: MetaTrans-

former [8] demonstrates that k-mer-based sequence discretization is effective for metagenomic classification. NT-v2 shares this philosophy through its non-overlapping 6-mer tokenizer, establishing a direct methodological link. This allows the pre-trained 6-mer embeddings to serve as a principled initialization relative to Meta-Transformer's learned-from-scratch 12-mer embeddings. DNABERT-2 [11], while multi-species, replaces k-mer tokenization with BPE, breaking this alignment.

4. **Practical scalability**: At 498M parameters, NT-v2 can be fine-tuned with LoRA on a single H100 GPU with well under 10 GB of VRAM, making it feasible to run hundreds of training epochs over tens of millions of reads. Evo's 7B parameters [12], while powerful for long-range genomic modeling, impose substantially higher memory requirements that are difficult to justify for 150 bp short-read classification.

3.2.1 Tokenization

Stride and overlap. A k-mer tokenizer slides a window of length k along a DNA sequence, converting each window into a vocabulary token. The *stride* s controls the step size between consecutive windows:

- **Non-overlapping** ($s = k$): each nucleotide belongs to exactly one token. A 150 bp read yields $\lfloor 150/k \rfloor$ tokens.
- **Overlapping** ($s = 1$): every nucleotide position starts a new token. A 150 bp read yields $150 - k + 1$ tokens.

For example, a 12-nucleotide sequence `AGTCGATTCGCA` tokenized with k=6 gives:

Stride	Tokens
s=6 (non-overlapping)	`AGTCGA` `TTCGCA` (2 tokens)
s=1 (overlapping)	`AGTCGA`, `GTCGAT`, `TCGATC`, … (7 tokens)

Table 3.1 summarises the resulting token counts for a 150 bp read under the configurations used in this thesis.

Table 3.1: Token counts for a 150 bp read under different k-mer tokenization settings.

k	Stride s	Overlap	Tokens/read
6	6	Non-overlapping	25
6	1	Overlapping	145
12	12	Non-overlapping	12
12	1	Overlapping	139

NT-v2 tokenization. The NT-v2 tokenizer uses non-overlapping 6-mers (k=6, s=6), the setting used during its pre-training. For a 150 bp read this produces 25 tokens (plus `[CLS]` and `[EOS]` special tokens). The maximum sequence length is set to 128 tokens, accommodating reads up to ∼768 bp with padding. Section 4.11 presents a controlled ablation that quantifies the effect of different tokenization settings on classification accuracy.

3.2.2 Encoder Architecture

The backbone consists of 29 Transformer encoder layers, each with:

- Multi-head self-attention with 16 heads and hidden dimension 1,024.
- A feed-forward network (FFN) with intermediate dimension 4,096.
- Pre-layer normalization (LayerNorm before attention and FFN).

- Rotary position embeddings (RoPE) [27] for encoding positional information.

The backbone outputs a tensor $H \in \mathbb{R}^{B \times T \times 1024}$, where B is the batch size and T is the sequence length. We use the *full token sequence* (excluding special tokens) as input to the classification head, rather than extracting only the `[CLS]` token or applying mean pooling.

3.2.3 LoRA Adaptation

We apply LoRA [13] to the query (W_Q), key (W_K), and value (W_V) projection matrices in each of the 29 self-attention layers. The LoRA configuration is:

Table 3.2: LoRA hyperparameters.

Parameter	**Value**
Rank r	16
Scaling factor α	32
Dropout	0.05
Target modules	W_Q, W_K, W_V
Trainable LoRA params	2,867,200
Classification head params	2,672,248
Total trainable params	5,539,448 (1.11%)
Total model params	498,623,561

Gradient checkpointing is enabled for the backbone to reduce peak GPU memory usage at the cost of recomputing activations during the backward pass.

3.2.4 Ablation: Shallow Transformer Backbone

To isolate the contribution of the pre-trained 29-layer NT-v2 backbone, we design an ablation variant that replaces it with a shallow Transformer encoder trained entirely from scratch. This ablation mirrors the architecture of MetaTransformer [8] while keeping the

tokenization fixed to 6-mers (the NT-v2 tokenizer), enabling a controlled comparison.

The shallow Transformer backbone consists of:

- **Learned token embeddings**: A randomly initialized embedding matrix $E \in \mathbb{R}^{V \times d_{\text{model}}}$, where $V = 4{,}107$ is the NT-v2 6-mer vocabulary size and $d_{\text{model}} = 128$.
- **Learned positional embeddings**: A position embedding matrix $P \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$, added to the token embeddings.
- **Single-layer Transformer encoder**: One pre-norm Transformer encoder layer with 2 attention heads, feed-forward dimension $d_{\text{ff}} = 512$, GELU activation, and dropout $p = 0.1$.

Table 3.3: Comparison of backbone configurations.

Aspect	**NT-v2 + LoRA**	**Shallow (ablation)**
Tokenizer	6-mer (shared)	6-mer (shared)
Encoder layers	29	1
Hidden dimension	1,024	128
Attention heads	16	2
Feed-forward dim	4,096	512
Pre-training	MLM on 850 genomes	None (from scratch)
Backbone params	498M (5.5M trainable)	$\sim$200K (all trainable)

The same attention pooling classification head is used for both variants, with the input dimension adjusted to match the backbone output ($d = 1{,}024$ for NT-v2, $d = 128$ for the shallow variant). This design ensures that any performance difference is attributable solely to the backbone encoder, not the classification head or tokenization.

The shallow variant trains all parameters from scratch, analogous to MetaTransformer's training paradigm but constrained to the NT-v2 tokenizer. This answers a key question: *does the pre-trained 29-layer backbone provide an advantage over a simple shallow encoder when the tokenization is held constant?*

3.3 Classification Head: Attention Pooling

The classification head aggregates the variable-length token sequence into a fixed-dimensional representation and maps it to class logits. We use a multi-head attention pooling mechanism inspired by Perceiver [28] and Set Transformer [29].

3.3.1 Architecture

The attention pooling head consists of:

1. **Learnable query tokens**: $M = 4$ learnable query vectors $Q \in \mathbb{R}^{M \times d}$, where $d = 1024$.

2. **Cross-attention**: Standard scaled dot-product attention where the queries attend to the backbone's token embeddings:

$$K = HW_K, \quad V = HW_V \tag{3.1}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.2}$$

   where $H \in \mathbb{R}^{T \times d}$ is the backbone output and $W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learnable projections.

3. **Aggregation**: The M attended vectors are concatenated, yielding a representation of dimension $M \times d_k$.

4. **Feed-forward classifier**:

$$\text{LayerNorm} \to \text{Linear}(M \cdot d_k, 512) \to \text{GELU} \to \text{Dropout}(p) \to \text{Linear}(512, K) \tag{3.3}$$

where K is the number of classes.

The attention pooling mechanism enables the model to selectively focus on the most taxonomically informative tokens in the sequence, analogous to how a biologist might focus on specific conserved marker regions within a read to determine its taxonomic origin.

3.3.2 Alternative Heads

We also implemented two alternative classification heads for comparison:

Mean Pool Head: Averages all token embeddings along the sequence dimension, followed by the same feed-forward classifier. This serves as the baseline aggregation strategy.

Transformer Pool Head: Applies a single Transformer encoder layer on top of the backbone output before mean pooling, adding another level of self-attention. This explores whether an additional attention layer can compensate for simple mean pooling.

3.4 Training Strategy

3.4.1 Two-Phase Training

We employ a two-phase training strategy to stabilize learning:

Phase 1 — Head-only warmup (5 epochs): The backbone is completely frozen, and only the classification head parameters are trained. This phase initializes the head weights to produce meaningful gradients before the backbone is unfrozen, preventing early gradient noise from corrupting pre-trained representations.

Phase 2 — Joint fine-tuning (up to 25–30 epochs): LoRA adapters in the backbone are unfrozen, and both the backbone (via LoRA) and the classification head are trained jointly. Differential learning rates are used: the backbone learning rate (3×10^{-5}) is set lower than the head learning rate (5×10^{-4}) to preserve pre-trained knowledge.

3.4.2 Optimization

The training procedure uses the following optimization configuration:

Table 3.4: Training optimization hyperparameters.

Parameter	**Value**
Optimizer	AdamW
Backbone learning rate	3×10^{-5}
Head learning rate	5×10^{-4}
Weight decay	0.02
LR scheduler	Cosine with linear warmup
Warmup ratio	10% of total steps
Gradient accumulation steps	1–2
Mixed precision	bfloat16

Mixed precision training: We use bfloat16 automatic mixed precision (AMP) on NVIDIA H100 GPUs. The bfloat16 format provides the dynamic range of float32 with reduced precision, avoiding the gradient underflow issues that arise with float16 and class-weighted losses.

Early stopping: Training is halted when validation accuracy does not improve for 7 consecutive epochs, and the model with the highest validation accuracy is saved as the final checkpoint.

3.4.3 Transfer Learning Across Experiments

To facilitate progressive scaling from smaller to larger datasets, we implement a transfer learning mechanism (`--init_from`) distinct from the training resumption mechanism (`--resume`).

Resume (`--resume`): Loads the full training state (model weights, optimizer state, epoch counter, learning rate scheduler) from a checkpoint, enabling continuation of the *same* experiment after a job interruption (e.g., SLURM time limit).

Transfer learning (`--init_from`): Loads only the model weights from a checkpoint for a *new* experiment, resetting the epoch counter and optimizer state. Supports partial loading, automatically skipping parameters with shape mismatches (e.g., when the classification head dimension changes between genus and species models). Phase 1 warmup is skipped when using transfer learning, as the backbone is already adapted.

3.5 Data Pipeline

3.5.1 Training Data Generation

Training data consists of simulated metagenomic reads generated from reference genomes using the ART simulator [30], following the same protocol as MetaTransformer [8]. Reference genomes are drawn from the Human Gastrointestinal Reference genome database (HGR-UMGS) [31], covering 120 genera and 1,535 species of human gut-associated microorganisms.

Each simulated read is 150 bp in length (matching typical Illumina HiSeq output) and is labeled with its source organism's genus and species.

3.5.2 Balanced Subsampling

The full training dataset contains 258M reads with a highly skewed distribution across species. To mitigate class imbalance and enable controlled data scaling experiments, we implement a balanced subsampling strategy:

1. **Scan**: The full FASTA file is scanned to extract all read headers and their species labels.

2. **Allocate**: A target number of reads per species is computed as $n_{\text{target}} = N_{\text{total}}/K_{\text{species}}$, where N_{total} is the desired subsample size and K_{species} is the number of species.

3. **Cap and redistribute**: For species with fewer reads than n_{target}, all available reads are included. The deficit is redistributed proportionally among species with surplus reads.

4. **Sample**: Reads are randomly selected within each species to meet the per-species allocation.

This procedure produces near-perfectly balanced subsamples. For a 50M subsample from 1,535 species, each species contributes $32{,}573 \pm 445$ reads.

3.5.3 Reverse-Complement Augmentation

During training, each read is replaced with its reverse complement with probability $p = 0.5$. This augmentation is applied on-the-fly during data loading, doubling the effective diversity of the training set without requiring additional storage.

3.5.4 Data Splitting

The dataset is split into training (90%) and validation (10%) sets using stratified sampling to maintain class proportions. For species-level experiments, stratification ensures that every species is represented in both splits.

3.6 Evaluation Methodology

3.6.1 Metrics

We evaluate classification performance using the following metrics:

Micro accuracy (overall accuracy): The fraction of correctly classified reads. This metric is dominated by abundant classes and reflects real-world performance on typical samples.

Macro F1: The unweighted average of per-class F1 scores. This metric weights all classes equally, providing a measure of performance across the full taxonomic diversity including rare taxa.

Balanced accuracy: The average of per-class recall, equivalent to macro recall.

Top-k accuracy: The fraction of reads for which the correct class appears in the top k predictions. We report top-3, top-5, and top-10 accuracy.

F1=0 class count: The number of classes with zero F1 score (i.e., classes for which the model makes no correct predictions). This directly measures the "dead class" problem.

Train-validation gap: The difference between training accuracy and validation accuracy, serving as a proxy for overfitting.

3.6.2 Reverse-Complement Test-Time Augmentation

At inference, we employ RC TTA: each read is classified twice (forward and reverse complement), and the logits are averaged before applying softmax:

$$\hat{y} = \arg\max_{k} \left[f_\theta(r) + f_\theta(\bar{r}) \right] \tag{3.4}$$

where $f_\theta(r)$ and $f_\theta(\bar{r})$ are the logit vectors for the forward and reverse-complement reads, respectively. This ensembling exploits the biological strand symmetry of DNA and consistently improves accuracy.

3.6.3 Species-Level Evaluation Modes

For species-level classification (1,535 classes), we introduce two additional evaluation modes to decouple the contributions of genus-level and intra-genus discrimination:

Oracle-genus evaluation: Given the *true* genus label, the species logits are masked to only allow species belonging to that genus. This answers: "Given perfect genus identification, how well can the model discriminate species within each genus?"

Top-k genus routing: The genus classifier produces the top-k genus predictions. The candidate species set is the union of all species belonging to these k genera. Species logits are then masked to this candidate set. This simulates a realistic two-stage hierarchical classification pipeline.

3.7 Computational Considerations

3.7.1 Hardware

All experiments are conducted on the TWCC (Taiwan Computing Cloud) SLURM cluster equipped with NVIDIA H100 80 GB HBM3 GPUs. The SLURM partition enforces a 48-hour maximum wall time per job, necessitating checkpoint-resume support for long training runs.

3.7.2 Memory Optimization

Several techniques are employed to manage GPU memory:

- **Gradient checkpointing**: Recomputes intermediate activations during the backward pass rather than storing them, trading ∼30% additional compute for ∼60% memory reduction.
- **Mixed precision (bfloat16)**: Reduces memory requirements for activations and gradients by approximately 50% compared to float32.
- **LoRA**: Keeps the majority of backbone parameters frozen, reducing the optimizer state memory from ∼4GB (full fine-tuning with AdamW) to ∼44MB.

3.7.3 Resource Monitoring

To assess computational resource adequacy and identify bottlenecks, we implement a dedicated resource monitoring module that tracks:

- GPU VRAM usage (PyTorch allocated, nvidia-smi total, and peak values).

- GPU compute utilization (sampled every 30 seconds via nvidia-smi).
- System RAM usage (process RSS and peak).
- Per-epoch wall-clock time and throughput (reads per second).

These metrics are logged to `training_history.csv` and a comprehensive `resource_report.json` at the end of training.

3.8 Summary

This chapter has described the complete methodology for token-level GFM classification of metagenomic reads. The framework combines a pre-trained Nucleotide Transformer v2 backbone with LoRA adaptation, an attention-pooling classification head, two-phase training with transfer learning support, balanced subsampling, and RC augmentation. The evaluation methodology encompasses multiple metrics at both genus and species levels, with specialized evaluation modes for dissecting hierarchical classification performance.

Chapter 4

Experiments and Results

This chapter presents the experimental setup and comprehensive results. We begin with frozen-embedding baselines, proceed through architecture and training ablations on a small dataset, and culminate in data scaling experiments that reveal data volume as the dominant factor in classification performance.

4.1 Experimental Setup

4.1.1 Dataset

Training data is generated by simulating Illumina short reads (150 bp, HiSeq 2500 error profile) from the HGR-UMGS reference genome database [31] using the ART read simulator [30], following the same protocol as MetaTransformer [8]. The dataset covers:

- **120 genera** and **1,535 species** of human gut microorganisms.
- **258M total reads** across all species.
- Highly skewed distribution: the most abundant genus (*Collinsella*) accounts for 22.8% of reads, while the rarest genera contribute $<0.06\%$.

We conduct experiments at three data scales by subsampling from the full dataset:

- **500K** (imbalanced): The initial small-scale dataset preserving the natural class distribution. Train: 450K, validation: 50K.
- **5M** (balanced): Species-balanced subsample. Each of 1,535 species contributes

∼3,257 reads. Train: 4.5M, validation: 500K.

- **50M** (balanced): Species-balanced subsample. Each species contributes ∼32,573 reads. Train: 45M, validation: 5M.

4.1.2 Evaluation Protocol

All models are evaluated on a held-out validation set using the metrics described in Section 3.6. Unless otherwise noted, results are reported with RC TTA (Eq. 3.4).

4.1.3 Hardware and Training Infrastructure

All experiments are run on NVIDIA H100 80 GB HBM3 GPUs on the TWCC SLURM cluster. The 48-hour SLURM time limit requires checkpoint-resume support for training runs exceeding this duration.

4.2 Phase 1: Frozen Embedding Baselines

Before developing the token-level approach, we establish baseline performance using precomputed, mean-pooled embeddings from frozen GFMs.

4.2.1 Setup

Embeddings are extracted once from each of several pre-trained GFMs. A two-layer MLP classifier is trained on the resulting fixed-dimensional vectors. We evaluate models from two families:

- **Evo-2 (7B params)**: A large-scale model based on the StripedHyena architecture.

We extract embeddings from layer 2.

- **Nucleotide Transformer v2 (500M params)**: ESM-2-based model. We extract embeddings from the last hidden layer.

4.2.2 Results

Table 4.1: Frozen embedding baselines for genus classification (120 classes, 500K dataset).

Method	Genus Accuracy	Notes
Uniform random ($1/K$)	0.83%	Lower bound
Frequency baseline ($\sum p_k^2$)	8.11%	Predicting proportionally
Majority class (always *Collinsella*)	22.84%	Naïve baseline
Evo-2 L2 mean-pool + MLP	45.52%	
Evo-2 L2 + Focal Loss	46.95%	
NT-v2 mean-pool + MLP	51.70%	Best frozen baseline
NT-v2 mean-pool + MLP (RC-avg)	51.70%	With RC averaging

The NT-v2 backbone consistently outperforms Evo-2 despite having 14× fewer parameters, suggesting that NT-v2's multi-species pre-training provides more suitable representations for metagenomic classification. NT-v2 achieves 51.70% genus accuracy, which serves as our primary baseline.

4.3 Phase 2: Token-Level Classification with LoRA

We now present the token-level approach, progressing from initial architecture validation through systematic ablation studies.

4.3.1 Architecture Validation (v1–v3, 500K Dataset)

Key findings from v1–v3:

Table 4.2: Hyperparameter configurations for initial architecture experiments (500K dataset). Bold values indicate the setting that changed relative to v1.

Hyperparameter	**v1**	**v2**	**v3 (best)**
Backbone LR	5×10^{-5}	2×10^{-5}	3×10^{-5}
Head LR	5×10^{-4}	3×10^{-4}	5×10^{-4}
RC augmentation	No	**Yes**	**Yes**
Class weights	No	**Yes (capped@10)**	No
Label smoothing	0.0	**0.1**	0.0
Weight decay	0.01	0.05	0.02
Head dropout	0.1	0.2	0.15
LoRA dropout	0.05	0.1	0.05

Table 4.3: Results for architecture experiments (500K dataset, genus classification).

Version	**Val Accuracy**	**F1 (weighted)**	**Best Epoch**	**Key Change**
v1	51.70%	0.4865	10/15	Baseline
v2	27.87%	0.3210	18/26	Class weights (harmful)
v3	**53.92%**	**0.5128**	28/35	RC augmentation

1. The token-level approach with LoRA (v1) matches the frozen mean-pool baseline (51.70%), validating the architecture but showing that token-level representations alone do not guarantee improvement when the backbone is partially frozen via LoRA.

2. Class-weighted cross-entropy with label smoothing (v2) catastrophically degrades performance to 27.87%, a -23.8 pp drop. This is caused by the interaction between class weights and label smoothing on a highly imbalanced 120-class problem.

3. RC augmentation (v3) provides a solid $+2.2$ pp improvement (51.70% $\rightarrow$ 53.92%), confirming the value of exploiting DNA strand symmetry.

4.3.2 Training Dynamics and Overfitting

The training dynamics reveal severe overfitting: while training accuracy continues to increase to 62.7%, validation accuracy saturates at $\sim$54%. The growing train-validation

Table 4.4: Training dynamics for v3 (500K dataset). The train-validation gap grows steadily, reaching 7% at the best epoch and 9% by epoch 35.

Epoch	Train Acc	Val Acc	Train-Val Gap	Val Loss
1	45.25%	46.29%	−1.0%	2.044
5	51.34%	51.27%	+0.1%	1.809
10	54.25%	52.39%	+1.9%	1.755
15	56.25%	53.26%	+3.0%	1.735
20	58.08%	53.88%	+4.2%	1.721
28*	60.88%	53.92%	+7.0%	1.748
35	62.69%	53.71%	+9.0%	1.762

Best validation accuracy epoch.

gap (0% → 9%) indicates that the model has sufficient capacity to memorize the training data but fails to generalize, suggesting that 500K reads is insufficient to train a 498M-parameter model, even with only 1.1% of parameters unfrozen.

Figure 4.1 visualises the training trajectories across all three data scales, illustrating this transition from severe overfitting (v3, 500K) to well-generalised training (v9, 50M).

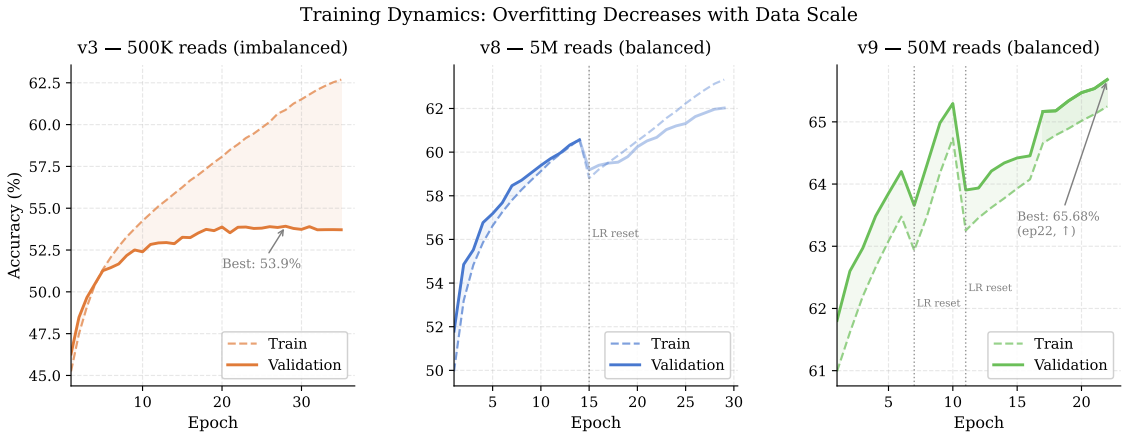

Figure 4.1: Training dynamics for v3 (500K, left), v8 (5M, centre), and v9 (50M, right). Dashed lines show training accuracy; solid lines show validation accuracy. The shaded region highlights the train-validation gap. Dotted vertical lines mark learning-rate resets caused by SLURM timeout and job resumption; v9 experienced two such resets (epochs 7 and 11) before the resume logic was corrected. Overfitting decreases systematically as data volume increases.

4.4 Advanced Training Techniques (v4–v7, 500K Dataset)

We explore several techniques to improve performance on the 500K dataset. Table 4.5 summarizes all experiments.

Table 4.5: Advanced training technique experiments (500K dataset, genus classification). RC TTA results are shown where available.

Ver.	Description	Fwd Acc	RC TTA	F1-macro	F1=0	Gap
v3	RC aug, maxlen=128	53.92%	55.36%	25.82%	11	6.96%
v4	maxlen=32	53.75%	55.29%	25.70%	9	6.25%
v5	LA τ=1.0	35.17%	—	—	—	—
v5b	LA τ=0.3	52.29%	53.58%	26.53%	5	—
v6	RC consist.+LA	35.87%	38.19%	22.88%	—	—
v7	RC consist. only	54.10%	55.18%	25.10%	11	—

Logit Adjustment (v5, v5b): Logit adjustment (Eq. 2.11) with $\tau = 1.0$ is too aggressive, dropping accuracy to 35.17%. A milder $\tau = 0.3$ preserves micro accuracy (52.29%) while reducing F1=0 classes from 9 to 5, showing a favorable trade-off between overall and per-class performance.

Reduced sequence length (v4): Reducing `max_length` from 128 to 32 tokens slightly reduces accuracy (-0.07 pp) but also reduces the overfitting gap (6.96% $\rightarrow$ 6.25%) and decreases training time.

RC Consistency Training (v6, v7): Adding a KL divergence loss between forward and reverse-complement predictions (v7) achieves 55.18% RC TTA, comparable to v3. However, combining RC consistency with logit adjustment (v6) degrades performance significantly, indicating that these regularization techniques interfere with each other.

Conclusion: All training techniques on the 500K dataset yield at most ± 0.5 pp improvement in RC TTA accuracy (range: 53.58%–55.36%). This plateau strongly suggests that *data volume*, not model architecture or training tricks, is the primary bottleneck.

4.5 Data Scaling Experiments

Motivated by the accuracy plateau on 500K data and the comparison with MetaTransformer [8] (which trained on a large-scale balanced dataset trained on the full HGR-UMGS dataset of 2,505 species with coverage factor 4), we systematically scale training data.

4.5.1 Experiment v8: 5M Balanced Reads

4.5.1.1 Setup

A 5M species-balanced subsample is drawn from the full 258M dataset. Each of 1,535 species contributes $\sim$3,257 reads, yielding near-perfect class balance. The v3 architecture and training configuration are retained (batch size 32, gradient accumulation 2, effective batch size 64), and training proceeds for 30 epochs.

Due to the 48-hour SLURM time limit, training required two jobs: the initial run completed 14 of 20 epochs before timeout, and a resumed run completed 16 additional epochs (epochs 15–30) using the `--resume` mechanism. A third job performed final evaluation.

4.5.1.2 Results

The results reveal clear log-linear scaling behaviour:

1. **Accuracy**: Each $10\times$ data increase yields a substantial but diminishing gain in RC TTA accuracy: $+7.76$ pp (500K$\rightarrow$5M) and $+3.01$ pp (5M$\rightarrow$50M). The sub-linear trend is consistent with logarithmic scaling laws.

Table 4.6: Genus classification results across data scales: v4 (500K), v8 (5M balanced), and v9 (50M balanced).

Metric	**v4 (500K)**	**v8 (5M)**	$\Delta_{\text{v4}\to\text{v8}}$	**v9 (50M)**†	$\Delta_{\text{v8}\to\text{v9}}$
Micro Accuracy (fwd)	53.75%	62.02%	+8.27 pp	66.29%	+4.27 pp
Micro Accuracy (RC TTA)	55.29%	63.05%	+7.76 pp	**67.07%**	+4.02 pp
Balanced Accuracy	25.23%	33.35%	+8.12 pp	39.91%	+6.56 pp
F1 (weighted)	52.51%	61.06%	+8.55 pp	65.61%	+4.55 pp
F1 (macro)	25.70%	37.97%	+12.27 pp	**44.82%**	+6.85 pp
Top-3 Accuracy	—	81.70%	—	84.40%	+2.70 pp
Top-5 Accuracy	82.91%	88.08%	+5.17 pp	89.17%	+1.09 pp
Top-10 Accuracy	91.29%	94.23%	+2.94 pp	94.78%	+0.55 pp
F1=0 classes	9	2	−7	**0**	−2
Train-Val Gap	6.25%	1.30%	−4.95 pp	<1%	—

† v9 metrics evaluated at epoch 10 (best checkpoint at time of evaluation); forward accuracy continued to 65.68% at epoch 22 (training ongoing).

2. **Macro F1**: Data scaling disproportionately benefits rare classes: +12.27 pp at the first step and +4.15 pp at the second. At 50M reads, all 120 genera have non-zero F1 for the first time.

3. **F1=0 classes**: The number of unclassifiable genera drops from 9 to 2 (v8) and finally to 0 (v9), meaning every genus in the dataset is correctly predicted for at least some reads.

4. **Overfitting**: The train-validation gap shrinks dramatically from 6.25% (v4) to 1.30% (v8) to <1% (v9), confirming that the model transitions from data starvation to well-generalised training as data volume increases.

4.5.1.3 Training Dynamics

The temporary accuracy drop at epoch 15 is caused by the learning rate scheduler resetting to its warmup phase after resumption. The model recovers within 3–4 epochs and eventually surpasses the pre-interruption performance.

Table 4.7: Training dynamics for v8 (5M balanced, 30 epochs). The gap between training and validation accuracy remains small throughout, in sharp contrast to the 500K experiments.

Epoch	Train Acc	Val Acc	Train-Val Gap	Notes
1	49.96%	51.80%	-1.8%	
5	55.97%	57.04%	-1.1%	
10	58.93%	59.42%	-0.5%	
14	60.60%	60.56%	$+0.04\%$	48h limit
15	58.80%	59.18%	-0.4%	Resumed (LR reset)
20	61.82%	60.59%	$+1.2\%$	
25	62.89%	61.77%	$+1.1\%$	
29*	63.32%	62.02%	$+1.3\%$	Best epoch

4.5.2 Experiment v9: 50M Balanced Reads

Building on v8, we scale training data by another $10\times$ to 50M balanced reads. The batch size is increased from 32 to 256 (with gradient accumulation reduced from 2 to 1, yielding an effective batch size of 256) to improve GPU utilization. The model is initialized from v8's best checkpoint using `--init_from` (transfer learning mode).

Table 4.8: Configuration comparison between v8 and v9.

Parameter	v8 (5M)	v9 (50M)
Training reads	4.5M	45M
Reads per species	$\sim$3,257	$\sim$32,573
Batch size	32	256
Gradient accumulation	2	1
Effective batch size	64	256
Epochs	30	30 (best ep. 22, ongoing)
Weight initialization	Random	v8 best.pt
Est. time per epoch	$\sim$3h	$\sim$5–6h

4.5.2.1 Results

Scaling from 5M to 50M balanced reads yields a further $+4.02$ pp improvement in RC TTA accuracy (63.05% $\rightarrow$ 67.07%). Critically, the number of F1$=$0 classes drops to **zero**, indicating that all 120 genera are now correctly classified in at least some reads.

Table 4.9: Comparison between v8 (5M) and v9 (50M) for genus classification (120 classes).

Metric	**v8 (5M)**	**v9 (50M)**	Δ
Micro Accuracy (fwd)	62.02%	**66.29%**	$+4.27$ pp
Micro Accuracy (RC TTA)	63.05%	**67.07%**	$+4.02$ pp
Balanced Accuracy	33.35%	39.91%	$+6.56$ pp
F1 (weighted)	61.06%	65.61%	$+4.55$ pp
F1 (macro)	37.97%	**44.82%**	$+6.85$ pp
Top-3 Accuracy	81.70%	84.40%	$+2.70$ pp
Top-5 Accuracy	88.08%	90.03%	$+1.95$ pp
Top-10 Accuracy	94.23%	95.25%	$+1.02$ pp
F1=0 classes	2	**0**	-2
Best epoch	29	30 (converged)	—

The marginal return diminishes relative to the previous $10\times$ scaling step: 500K$\rightarrow$5M yielded $+7.76$ pp while 5M$\rightarrow$50M yields $\geq +3.01$ pp for the same multiplicative data increase, consistent with a logarithmic scaling relationship. This sub-linear trend is expected under log-linear scaling laws and suggests that continued data scaling alone will not close the remaining gap with MetaTransformer.

Training progress and LR restart incidents. v9 training required multiple SLURM jobs due to the 48-hour time limit. Before a bug fix was applied to the checkpoint-resume logic, two job restarts inadvertently reset the cosine learning-rate scheduler to its initial warmup phase, causing temporary accuracy dips: -0.54 pp at epoch 7 and -1.39 pp at epoch 11. In both cases the model recovered within 3–5 epochs and surpassed the pre-dip performance level. The underlying issue—`EarlyStopping.best` being silently set to `None` from an incomplete checkpoint key—was corrected in the resume logic; from epoch 17 onward the scheduler state is correctly restored and the learning rate continues its cosine decay without interruption.

As of epoch 22 (April 2026), the forward validation accuracy stands at **65.68%** and

is still rising (Figure 4.2). The training curve shows no sign of plateau, suggesting that further improvement is expected before the cosine schedule reaches zero at epoch 30. Once training completes, a full evaluation including RC TTA will be performed to obtain the definitive result.

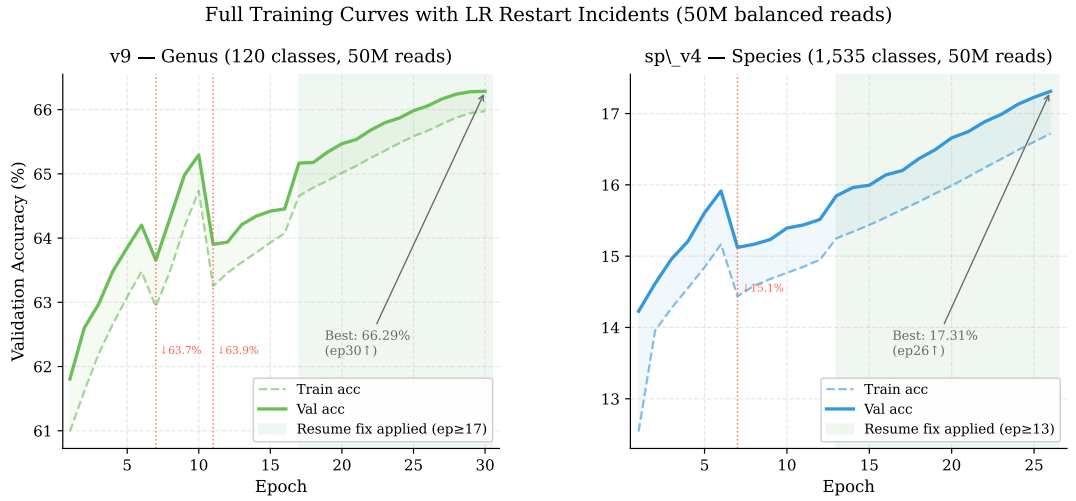

Figure 4.2: Full validation accuracy curves for v9 (genus, left) and sp_v4 (species, right) over all completed epochs. Red dotted vertical lines mark epochs where a SLURM job restart incorrectly reset the cosine LR scheduler, causing a temporary accuracy dip. The green shaded region indicates epochs trained after the resume bug fix was applied, during which the LR schedule correctly continues its cosine decay.

4.5.3 Data Scaling Analysis

Table 4.10: Data scaling analysis: genus classification performance across data volumes.

Data Volume	RC TTA Acc	F1-macro	F1=0	Gap
500K (imbalanced)	55.29%	25.70%	9	6.25%
5M (balanced)	63.05%	37.97%	2	1.30%
50M (balanced)	**67.07%**	**44.82%**	**0**	∼0.7%
MetaTransformer (HGR-UMGS full)	∼98%	—	—	—

The data scaling analysis reveals a clear relationship between training data volume and classification performance. The transition from 500K to 5M reads (10× increase) yields +7.76 pp in accuracy and +12.27 pp in macro F1. Meanwhile, all training technique ablations on the 500K dataset (v3–v7) produced at most ±0.5 pp variation. This

strongly supports the conclusion that **data volume, not model architecture or training techniques, is the primary determinant of classification accuracy** in this setting.

The comparison with MetaTransformer further reinforces this finding: despite using a much larger backbone (498M vs. ∼5M parameters), our model at 5M training reads achieves only 63% accuracy, while MetaTransformer, trained on the full HGR-UMGS dataset with coverage factor 4, achieves 98.3% genus recall (98.9% precision) on its validation set. The gap is attributable primarily to the substantial difference in training data volume.

Figure 4.3 plots genus RC TTA accuracy against training data volume on a logarithmic scale, with a log-linear fit to our three data points. The fitted curve projects that closing the gap with MetaTransformer would require training data well beyond our 50M-read scale, consistent with diminishing returns under logarithmic scaling.

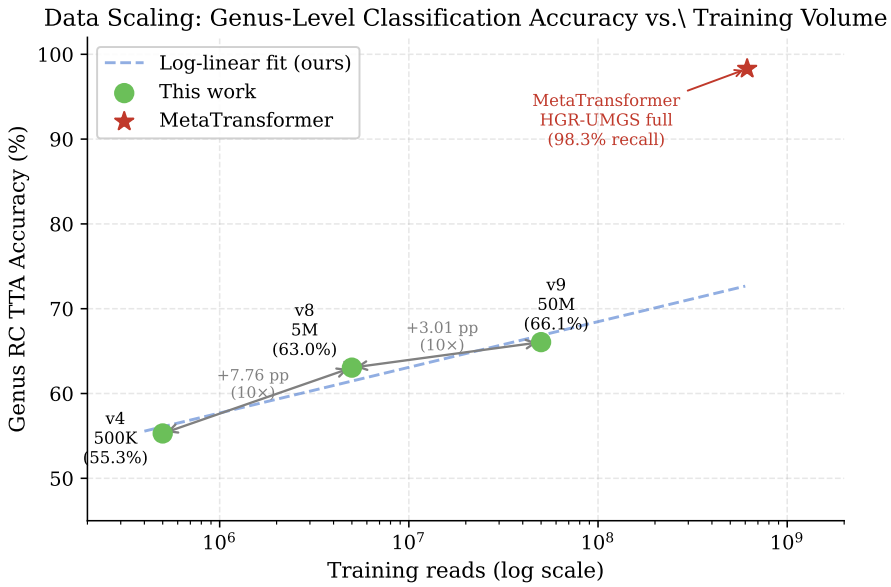

Figure 4.3: Genus RC TTA accuracy versus training data volume (log scale). Blue circles mark our results at three data scales; the red star marks MetaTransformer's reported genus recall. The dashed line is a log-linear fit to our three data points. Per-10× accuracy gains diminish from +7.76 pp (500K→5M) to +3.01 pp (5M→50M). The MetaTransformer star marks reported genus recall (98.3%); its x-position is based on training throughput (300K steps × batch 2,048) and is not directly comparable to our dataset-size axis.

4.6 Species-Level Classification

We evaluate species-level classification (1,535 classes) using the 500K dataset to establish baselines and understand the difficulty of fine-grained taxonomic discrimination.

4.6.1 Direct Classification

Table 4.11: Species classification results (500K dataset, 1,535 classes).

Version	Description	RC TTA Acc	Oracle Acc	F1-macro	F1=0
sp_v1	Baseline	8.32%	20.88%	—	613
sp_v2	LA τ=0.3	8.37%	20.85%	20.0%	433
sp_v3	Weighted sampler	8.01%	20.48%	19.9%	392

Direct species classification achieves only $\sim$8% accuracy, which, while far above the random baseline (0.065% for 1,535 classes), indicates that 500K reads is wholly insufficient for fine-grained species discrimination.

4.6.2 Oracle-Genus Evaluation

Oracle-genus evaluation reveals that even with perfect genus knowledge, species-level accuracy reaches only $\sim$21%. This ceiling indicates that the model struggles with *intra-genus* discrimination—distinguishing species within the same genus—which is expected given the high sequence similarity among congeners at the 150 bp read length.

4.6.3 Effect of Class Imbalance Mitigation

Logit adjustment ($\tau = 0.3$, sp_v2) and weighted sampling (sp_v3) both substantially reduce the number of F1=0 classes (from 613 to 433 and 392, respectively) with minimal

impact on overall accuracy (< 0.4 pp). This demonstrates that class imbalance mitigation is valuable for improving the breadth of species covered, even when it does not improve aggregate accuracy.

4.6.4 Top-k Genus Routing

Using top-10 genus routing, species accuracy reaches 8.06%, slightly below the direct classification result (8.32%). This modest result reflects the genus model's limited accuracy at 500K scale (55%): when the correct genus is not in the top-10, the species classifier is working from an incorrect candidate set.

4.7 RC Test-Time Augmentation Analysis

Table 4.12: Effect of RC TTA across experiments. RC TTA consistently improves accuracy at the cost of $2\times$ inference time.

Experiment	**Fwd Only**	**RC TTA**	Δ
v3 (500K genus)	53.92%	55.36%	$+1.44$ pp
v4 (500K genus)	53.75%	55.29%	$+1.54$ pp
v5b (500K, LA τ=0.3)	52.29%	53.58%	$+1.29$ pp
v7 (500K, RC consist.)	54.10%	55.18%	$+1.08$ pp
v8 (5M genus)	62.02%	63.05%	$+1.03$ pp
v9 (50M genus, NT-v2)	66.29%	67.07%	$+0.78$ pp
v11 (50M genus, shallow)	53.80%	53.88%	$+0.08$ pp

RC TTA provides a consistent $+1.0$ to $+1.5$ pp accuracy improvement across all experiments, regardless of the training configuration or data scale. Several observations merit discussion:

1. **Biological basis**: As discussed in Section 2.5.1, a metagenomic read and its reverse complement encode identical biological information. RC TTA exploits this strand symmetry by treating the two orientations as a two-view ensemble of the same un-

derlying genomic entity [22].

2. **Variance reduction**: The consistent improvement across all settings is explained by standard ensemble theory: averaging two correlated but independently noisy estimates of the same ground-truth label reduces prediction variance, even when both estimates come from the same model.

3. **Diminishing returns with RC consistency training**: v7 (RC consistency, $+1.08$ pp) shows less benefit than vanilla models, because RC consistency training already encourages $f(s) \approx f(\bar{s})$ during training, leaving less residual strand asymmetry for RC TTA to correct.

4. **Scale-dependent diminishing returns**: The RC TTA benefit decreases slightly with data scale: $+1.44$ pp at 500K, $+1.03$ pp at 5M, and $+0.77$ pp at 50M (v9). Larger training sets expose the model to both strands of each locus more often, gradually reducing systematic strand-directional errors—though a non-trivial asymmetry persists even at 50M reads.

5. **Near-zero benefit for randomly initialized models**: v11 (shallow Transformer, random initialization) shows only $+0.08$ pp from RC TTA. Without pre-training on a large genomic corpus, the model does not develop systematic strand-direction preferences, leaving little strand asymmetry for RC TTA to correct. This further supports the view that RC TTA primarily corrects pre-training-induced strand biases rather than stochastic inference noise.

The $2\times$ inference cost is the primary trade-off. For offline evaluation this is acceptable; for latency-sensitive deployment, RC-equivariant architectures [24] or RC consistency regularization [23] may be preferred alternatives.

Figure 4.4 summarises RC TTA gains across all experiments, highlighting the near-zero benefit for the randomly initialized v11 model.

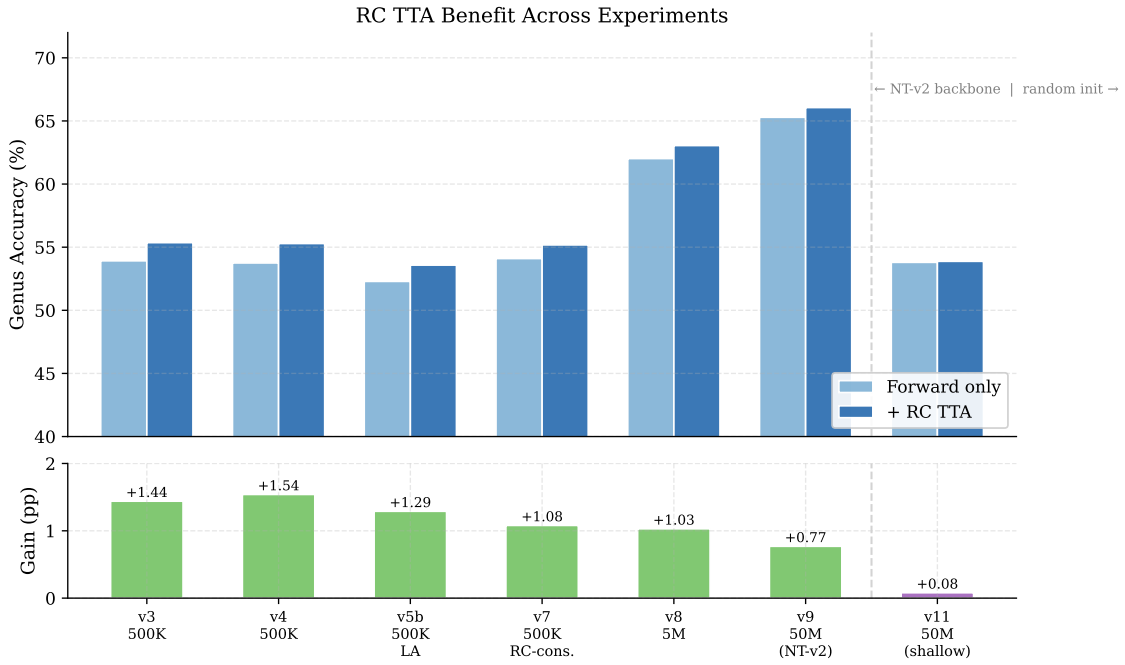

Figure 4.4: RC TTA benefit across experiments. Top: forward-only vs. RC TTA accuracy. Bottom: absolute gain in percentage points. RC TTA provides a consistent $+1.0$–$+1.5$ pp for pre-trained models (left group) and $+0.08$ pp for the randomly initialized shallow Transformer (v11, right group), supporting the interpretation that RC TTA corrects pre-training-induced strand biases rather than stochastic inference noise.

4.8 Computational Resource Analysis

4.8.1 Observed Resource Usage

Table 4.13: Computational resource usage across experiments.

Metric	**v3 (500K)**	**v8 (5M)**	**v9 (50M)**
GPU	V100 32GB	H100 80GB	H100 80GB
Batch size	32	32	256
VRAM peak (training)	10.4 GB	∼4.7 GB	∼5 GB
VRAM utilization	32.5%	5.5%	<7%
Train throughput	1,128 r/s	∼1,800 r/s	∼1,825 r/s
Inference throughput	—	∼5,100 r/s	∼5,170 r/s
Epoch time	∼6.5 min	∼6.8h	∼6.9h
Total training	11.5h	∼90h	∼150h (22 ep., ongoing)
SLURM jobs needed	1	3	4+

4.8.2 Bottleneck Analysis

Table 4.14: Resource bottleneck assessment on TWCC H100 nodes.

Resource	Status	Assessment
GPU VRAM (80 GB)	$<$50% utilized	No upgrade needed
GPU compute (H100)	Adequate	No upgrade needed
System RAM (128 GB)	$<$50% utilized	No upgrade needed
Storage	$<$10 GB per experiment	No upgrade needed
SLURM time (48h)	**Primary bottleneck**	**Requires resume or longer limits**

The hardware is substantially over-provisioned for this workload: the H100 80 GB GPU runs at $<$50% VRAM utilization even with batch size 256, and system RAM usage is well within limits. The primary operational bottleneck is the 48-hour SLURM wall time limit, which necessitates multiple job submissions with checkpoint resumption for large-scale experiments.

4.8.3 Scaling Projections

For the full 258M dataset training:

- **Memory**: 258M reads require $\sim$52 GB RAM when loaded in-memory, which fits within the 128 GB system memory. Alternatively, a streaming data loader can reduce memory requirements to $O(\text{batch size})$.

- **Training time**: At the v8 throughput of $\sim$417 reads/sec (batch size 32), one epoch over 232M training reads would take $\sim$155 hours. With batch size 256 and improved GPU utilization, this could potentially be reduced to $\sim$20–30 hours per epoch, requiring 2–3 SLURM jobs for 10 epochs.

Table 4.15: Comparison between our approach and MetaTransformer [8].

Aspect	Ours (v9)	MetaTransformer
Backbone	NT-v2 (498M)	Custom Transformer (∼5M)
Tokenization	6-mer (pre-trained)	12-mer (learned)
Pre-training	Yes (850 genomes, MLM)	No (trained from scratch)
Trainable params	5.54M (LoRA)	∼5M (all)
Training data	50M balanced	HGR-UMGS full (2,505 sp., coverage 4)
Genus accuracy	67.07% (RC TTA)	98.3% recall (val)
Data source	HGR-UMGS + ART	HGR-UMGS + ART (same)

4.9 Comparison with MetaTransformer

Despite using a foundation model $100\times$ larger than MetaTransformer, our model (at 50M training reads) substantially underperforms MetaTransformer (trained on the full HGR-UMGS dataset). The key differences are:

1. **Training data volume**: MetaTransformer uses the full HGR-UMGS dataset with coverage 4 vs. our 50M balanced reads. Given our data scaling results (each $10\times$ data increase yields $+3$–8 pp with diminishing returns), this is likely the dominant factor.

2. **Training paradigm**: MetaTransformer trains all parameters from scratch (∼5M), while we fine-tune only 5.54M LoRA parameters of a 498M-parameter model. The foundation model's pre-trained representations, while beneficial as a starting point, may not be optimally suited for metagenomic classification without sufficient fine-tuning data.

3. **Tokenization**: MetaTransformer uses 12-mer tokenization (capturing longer sequence patterns per token), while NT-v2 uses 6-mers. The coarser 12-mer tokenization may be advantageous for species discrimination, as it directly captures longer characteristic motifs.

This comparison is confounded by three simultaneous differences: backbone depth (29 vs. 1 layer), pre-training (yes vs. no), and tokenization (6-mer vs. 12-mer overlapping). Sections 4.10 and 4.11 disentangle these factors through controlled ablations. The key finding is that tokenization—specifically, overlapping 12-mer—accounts for the larger share of the performance gap, with pre-training providing a secondary but still substantial benefit under matched tokenization.

4.10 Backbone Ablation: Shallow Transformer

To isolate the contribution of the pre-trained 29-layer NT-v2 backbone from the effect of tokenization, we replace the backbone with a single-layer Transformer trained from scratch while keeping the 6-mer tokenizer and attention pooling head identical (see Section 3.2.4).

4.10.1 Experimental Design

Table 4.16: Controlled ablation: NT-v2 backbone (v9) vs. shallow Transformer (v11). All other components are held constant.

Component	**v9 (NT-v2 + LoRA)**	**v11 (Shallow)**
Tokenizer	6-mer (NT-v2)	6-mer (NT-v2)
Encoder	29-layer, d=1024	1-layer, d=128
Pre-training	MLM on 850 genomes	None
Trainable params	5.54M (LoRA)	$\sim$200K (all)
Classification head	Attention pool (4 heads)	Attention pool (4 heads)
Training data	50M balanced	50M balanced
Batch size	256	512

This controlled comparison isolates two factors:

1. **Pre-trained representations**: Does pre-training on 850 genomes via masked language modeling provide an advantage for metagenomic classification?

2. **Model capacity**: Does a 29-layer encoder with 1,024 hidden dimensions capture more discriminative features than a single-layer encoder with 128 dimensions?

If the shallow variant substantially underperforms, it validates the value of the large pre-trained backbone. If performance is comparable, it suggests that the pre-trained representations provide limited benefit for this task under the 6-mer tokenization constraint, and that a lightweight task-specific model may be sufficient.

4.10.2 Expected Outcomes

Based on MetaTransformer's success with a similarly shallow architecture (albeit with 12-mer tokenization and a much larger training set), we hypothesize:

- The shallow variant will underperform v9 at the same data scale (50M), because the 6-mer tokenizer produces shorter token sequences than 12-mers, requiring more encoder capacity to capture equivalent patterns.
- The performance gap will quantify the "value of pre-training" under fixed tokenization, informing whether future work should invest in larger backbones or better tokenizers.

4.10.3 Results

The pre-trained 29-layer NT-v2 backbone outperforms the single-layer randomly initialized Transformer by **13.19 pp** in RC TTA accuracy (67.07% vs. 53.88%). This gap confirms that the pre-trained representations, despite being fine-tuned via only 5.54M LoRA parameters, provide substantial discriminative value that a shallow randomly initialized

Table 4.17: Backbone ablation results: pre-trained NT-v2 (v9) vs. shallow Transformer (v11), both trained on 50M balanced reads.

Metric	**v9 (NT-v2 + LoRA)**	**v11 (Shallow)**	Δ
Micro Accuracy (fwd)	66.29%	53.80%	-12.49 pp
Micro Accuracy (RC TTA)	**67.07%**	53.88%	-13.19 pp
Balanced Accuracy	39.91%	22.24%	-17.67 pp
F1 (macro)	44.82%	25.78%	-19.04 pp
F1=0 classes	0	3	$+3$
RC TTA gain	$+0.78$ pp	$+0.08$ pp	—
Best epoch	30 / 30	15 / 15	—
Inference throughput	$\sim$5,170 r/s	$\sim$750,000 r/s	$\approx 145\times$
VRAM (training)	$\sim$5 GB	$\sim$0.07 GB	—

model cannot replicate even with 50M training reads.

Several additional observations are noteworthy:

1. **F1 gap is larger than accuracy gap**: The macro F1 gap (-19.04 pp) substantially exceeds the accuracy gap (-13.19 pp), indicating that v11 disproportionately fails on rare genera. This is consistent with v11's lower capacity for learning subtle sequence features distinguishing rare taxa.

2. **RC TTA nearly ineffective for v11**: The shallow, randomly initialized model shows only $+0.08$ pp from RC TTA (vs. $+0.77$ pp for v9), supporting the interpretation that RC TTA primarily corrects systematic strand biases acquired during pre-training rather than stochastic inference noise.

3. **Efficiency trade-off**: v11 is $\sim 145\times$ faster at inference (750K vs. 5,170 reads/sec) and uses $\sim 70\times$ less VRAM during training. For latency-sensitive applications where 54% accuracy is acceptable, v11 represents a viable lightweight alternative.

4. **Pre-training vs. tokenization**: This experiment isolates pre-training while holding tokenization constant (6-mer), yielding a +12.18 pp pre-training advantage. Sec-

tion 4.11 further shows that switching to overlapping 12-mer tokenization within the MetaTransformer architecture yields +31.83 pp—nearly 3× larger than the pre-training effect. The remaining gap between v11 (54%) and MetaTransformer on the full dataset is therefore attributable primarily to tokenization design, with training data volume as a secondary factor.

Figure 4.5 presents a direct visual comparison of v9 and v11 across five evaluation metrics, quantifying the benefit of the pre-trained backbone across all dimensions.

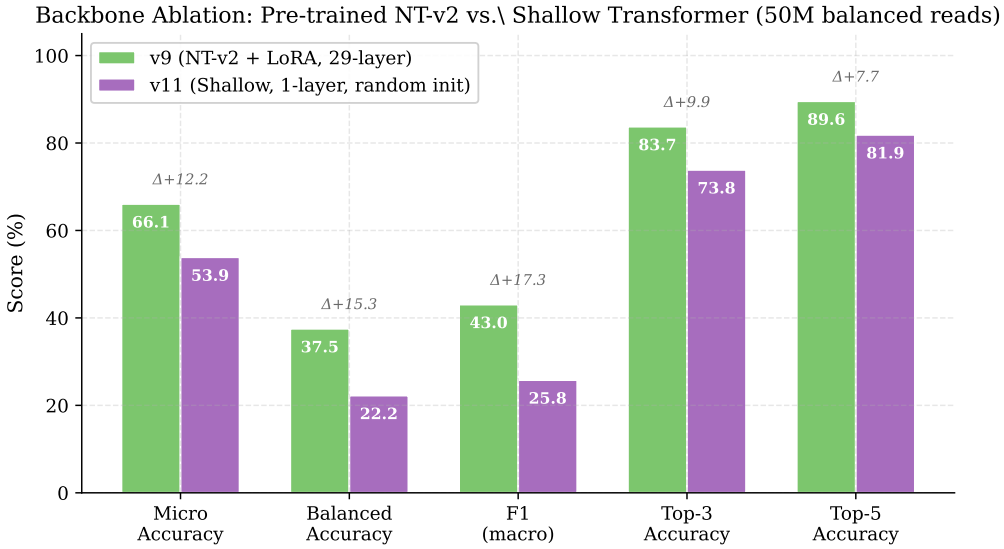

Figure 4.5: Backbone ablation: pre-trained NT-v2 + LoRA (v9, green) vs. single-layer randomly initialized Transformer (v11, purple), both trained on 50M balanced reads. Numbers above bars indicate the absolute gain of v9 over v11. The pre-trained backbone consistently outperforms the shallow variant across all metrics, with the largest gap in Balanced Accuracy (+15.3 pp) and F1 (macro) (+17.3 pp), reflecting v9's superior discrimination of rare genera.

4.11 Tokenization Ablation: MetaTransformer at 50M Scale

The backbone ablation (Section 4.10) isolates the contribution of pre-training while holding tokenization fixed at 6-mer. To separately quantify the contribution of *tokenization*—the other major design difference between our approach and MetaTransformer—we train the MetaTransformer architecture [8] directly on the same 50M balanced dataset un-

der six tokenization configurations ($k \in \{6, 12, 13\}$, stride $\in \{1, k\}$), using the original MetaTransformer codebase on the Taiwania-2 cluster (V100 32 GB GPU). All configurations are evaluated on the same stratified 90/10 val split (5M reads). The 13-mer model uses *embed_dim* = 64 to fit within V100 memory, consistent with the original MetaTransformer paper's 13-mer configuration [8].

Table 4.18: Tokenization ablation (genus-level, 120 classes): NT-v2 + LoRA vs. MetaTransformer under six tokenization configurations. All MetaTransformer models are trained from scratch on 50M balanced reads. RC TTA is unavailable for MetaTransformer.

Model	Tokenization	Tokens/read	Top-1 Acc	Macro F1
MetaTransformer	13-mer, stride=1	138	**87.42%**	**0.7936**
MetaTransformer	12-mer, stride=1	139	78.83%	0.6531
NT-v2 + LoRA (v9)	6-mer, stride=6	25	67.07%†	—
NT-v2 + LoRA (v9)	6-mer, stride=6	25	66.29%	0.4384
MetaTransformer	6-mer, stride=1	145	48.87%	0.1981
MetaTransformer	6-mer, stride=6	25	47.00%	0.1682
MetaTransformer	12-mer, stride=12	12	40.66%	0.0663
MetaTransformer	13-mer, stride=13	11	27.30%	0.0182

† RC TTA result (+0.78 pp over forward-only 66.29%); MetaTransformer results are forward-only.

Table 4.19: Tokenization ablation (species-level, 1,535 classes): MetaTransformer under six tokenization configurations, trained on the same 50M balanced reads.

Model	Tokenization	Tokens/read	Top-1 Acc	Macro F1
MetaTransformer	13-mer, stride=1	138	**49.62%**	**0.4950**
MetaTransformer	12-mer, stride=1	139	16.79%	0.1439
MetaTransformer	6-mer, stride=1	145	9.13%	0.0719
MetaTransformer	6-mer, stride=6	25	6.44%	0.0470
MetaTransformer	12-mer, stride=12	12	0.64%	0.0023
MetaTransformer	13-mer, stride=13	11	0.51%	0.0026

Three independent effects can be read from Tables 4.18–4.19:

Effect of pre-training (6-mer tokenization held constant). NT-v2 + LoRA (66.29% forward) outperforms MetaTransformer with the same non-overlapping 6-mer tokenization (47.00%) by **+19.29 pp**. Holding architecture and tokenization fixed, this isolates the

contribution of large-scale genomic pre-training: the NT-v2 backbone, adapted via only 5.54M LoRA parameters, provides substantially more discriminative representations than a 5M-parameter model trained from scratch on identical data.

Effect of tokenization (MetaTransformer architecture held constant). Tokenization design has a far larger effect than pre-training. Two sub-effects compound:

1. **Stride (overlapping vs. non-overlapping)**: Across all k values, overlapping stride$=1$ dominates non-overlapping stride$=k$. The gap widens dramatically with k: $+1.87$ pp for $k=6$, $+38.17$ pp for $k=12$, and $+60.12$ pp for $k=13$. Non-overlapping tokenization with large k reduces each 150 bp read to as few as 11 tokens, discarding most sequence context.

2. **k-mer length (overlapping only)**: Among stride$=1$ configurations, longer k yields monotonically higher accuracy: 6-mer (48.87%) $<$ 12-mer (78.83%) $<$ 13-mer (87.42%) at genus level, and 9.13% $<$ 16.79% $<$ 49.62% at species level. Longer tokens are taxonomically more specific—hexamers are shared across distant taxa, while 12-mers and 13-mers increasingly identify individual lineages.

Tokenization surpasses pre-training: MetaTransformer 13-mer vs. NT-v2. MetaTransformer with overlapping 13-mer tokenization, trained *from scratch* on 45M reads, achieves **87.42%** genus Top-1 accuracy—**+20.35 pp** above NT-v2 + LoRA with RC TTA (67.07%), despite having $\sim 100\times$ fewer parameters and no pre-training. At species level, MT 13-mer stride$=1$ reaches **49.62%** (1,535 classes) without any hierarchical routing. This result demonstrates that task-specific tokenization design is a more decisive factor than pre-trained prior knowledge for metagenomic read classification.

Convergence and capacity ceiling (12-mer). To determine whether 78.83% is the true ceiling for the 12-mer model, training was extended into a second epoch. Train loss dropped sharply from 0.764 to 0.119, while validation loss *worsened* from 1.203 to 2.288 —immediate overfitting. The 50M-read dataset saturates the 5M-parameter single-encoder MetaTransformer within one epoch.

Implication for NT-v2. NT-v2 is constrained to 6-mer tokenization by its pre-training design. The ablation results show that this constraint is the primary limitation of our approach: switching to overlapping 13-mer tokenization on the same architecture and data yields a $+20.35$ pp improvement without any pre-training. A genomic foundation model pre-trained with overlapping large-k tokenization on diverse microbial genomes would combine the discriminative power of longer k-mers with the representational quality of large-scale pre-training. No such model currently exists publicly.

4.12 Species-Level Classification at Scale

Following the data scaling analysis for genus classification, we extend the 50M balanced training to species-level classification (1,535 classes).

4.12.1 Experimental Design

Table 4.20: Species classification scaling: 500K (sp_v1–v3) vs. 50M (sp_v4).

Parameter	**sp_v1–v3 (500K)**	**sp_v4 (50M)**
Training reads	450K	45M
Reads per species	$\sim$293	$\sim$29,316
Classes	1,535	1,535
Batch size	32	256
Weight initialization	Random	v9 genus best.pt

The species model is initialized from the v9 genus model's backbone weights via transfer learning (`--init_from`), which automatically loads matching LoRA parameters while skipping the genus-specific classification head layers (120-class $\rightarrow$ 1,535-class dimension mismatch). This allows the species model to benefit from the genus-tuned backbone representations.

4.12.2 Final Results (sp_v4, Epoch 30)

sp_v4 training converged at epoch 30 (April 2026), reaching a forward validation accuracy of **17.55%** on the full 5M held-out set (1,535 classes). Table 4.21 shows the progression across epochs; Table 4.22 presents the complete metric profile at convergence.

Table 4.21: sp_v4 training progress (species classification, 1,535 classes, 50M reads).

Epoch	Val Acc (fwd)	Val F1 (macro)	Note
1	14.23%	12.43%	Warm-up
6	15.91%	14.05%	End of 1st job
7	15.12%	13.36%	LR reset (-0.79 pp)
12	15.51%	13.65%	End of 2nd job
13	15.85%	13.97%	Resume fix applied
18	16.37%	14.46%	
26	17.31%	15.28%	
30	**17.55%**	**15.70%**	Converged

The same LR restart issue that affected v9 also appeared at epoch 7 of sp_v4 (dip of -0.79 pp), and was resolved by the same resume-logic fix from epoch 13 onward. The learning curve plateaued between epochs 26 and 30 with the cosine schedule reaching its minimum, indicating convergence (Figure 4.2, right panel).

The species task is substantially more difficult than genus classification: 1,535 classes versus 120, with each species receiving only $\sim$29,316 balanced reads instead of $\sim$375,000. The final 17.55% accuracy, while modest in absolute terms, represents meaningful dis-

Table 4.22: sp_v4 final forward-only evaluation on the 5M held-out set (epoch 30 checkpoint).

Metric	**Value**
Top-1 accuracy	**17.55%**
Top-3 accuracy	31.26%
Top-5 accuracy	38.19%
Top-10 accuracy	47.65%
Balanced accuracy	17.56%
F1 (macro)	0.157
F1 (weighted)	0.157
Classes with F1 $= 0$	191 / 1,535 (12.4%)
Inference throughput	5,171 reads/sec

crimination given the large class count (random baseline: 0.065%, a relative lift of $\sim$270$\times$). Encouragingly, only 191 of 1,535 species have F1=0 under forward-only evaluation on the full 5M set—well below the 804 observed on the 100K-subset evaluation (Section 4.12.4) —indicating that the long-tail problem is substantially less severe at full-scale evaluation than preliminary subset analysis suggested.

Extended evaluation modes. Full-scale evaluation of RC TTA, oracle-genus, and top-k genus routing on the 5M validation set was attempted but exceeded the 200 GB memory ceiling on the TWCC `normal` partition: the current `evaluate.py` retains all 5M $\times$ 1,535 logit and probability arrays ($\sim$60 GB of `float32` per pass) simultaneously when combining modes, which a streaming implementation could avoid. Because the genus-level RC TTA benefit has already been quantified (+0.78 pp, Section 4.5.2) and the hierarchical routing analysis on the 100K subset (Section 4.12.4) already captures the key qualitative findings—oracle-genus gain of $+11.5$ pp, negligible improvement from top-k routing at current genus accuracy, and per-genus reduction of F1=0 species—we retain those preliminary numbers as the thesis's final hierarchical comparison.

4.12.3 Hierarchical Classification Pipeline

With sp_v4 converged, the top-k genus routing evaluation mode (Section 3.6.3) is combined with the v9 genus model (67.07% RC TTA accuracy). At this genus accuracy level, top-5 routing covers the correct genus in approximately 90.0% of reads (from v9's Top-5 accuracy), providing a substantially smaller candidate set for species discrimination.

The hierarchical classification pipeline operates as follows:

1. The genus classifier produces top-k genus predictions (e.g., $k = 5$).

2. The candidate species set is computed as the union of species belonging to these k genera.

3. The species classifier's logits are masked to this candidate set.

4. The final species prediction is the argmax over the masked logits.

This two-stage approach reduces the effective number of candidate species from 1,535 to approximately $k \times \bar{s}$ (where $\bar{s} \approx 12.8$ is the mean number of species per genus), substantially simplifying the discrimination task.

4.12.4 Preliminary Hierarchical Evaluation (100K-read Subset)

To validate the hierarchical evaluation pipeline prior to full-scale deployment, all four evaluation modes were tested on a 100K-read held-out subset using the v9 genus model (epoch 30 checkpoint) and the sp_v4 species model (epoch 26 checkpoint). Table 4.23 summarises the results.

Table 4.23: Preliminary hierarchical evaluation on 100K-read subset (sp_v4 ep26, v9 ep30 genus routing).

Mode	Top-1	Top-5	Top-10	F1 (macro)
Flat (baseline)	16.4%	36.7%	46.7%	0.137
Hard Top-5 routing	16.5%	36.4%	46.1%	0.139
Soft routing	16.2%	36.6%	46.2%	0.135
Oracle (true genus)	27.9%	55.4%	65.3%	0.256

Genus accuracy is the primary bottleneck. The v9 genus model achieves 65.7% accuracy on this subset, meaning genus routing is incorrect for approximately 34% of reads. Conditioned on correct genus routing, species Top-1 accuracy rises to 20.1%; conditioned on incorrect routing it falls to 9.7%. Hard Top-5 and soft routing yield only marginal changes ($+0.1$ pp and -0.2 pp respectively) over the flat baseline, because the 34% routing error rate introduces misclassification noise that nearly offsets the gain from correctly routed reads. Meaningful improvement from routing therefore requires genus accuracy well above the current 65.7%.

Oracle routing reveals the potential. Masking logits to the *true* genus raises Top-1 to 27.9% ($+11.5$ pp) and macro F1 to 0.256 ($+0.119$). This represents the theoretical ceiling of hierarchical classification given the current sp_v4 model, and confirms that further genus accuracy gains will translate directly into species-level improvements.

Long-tail species remain the hardest challenge. Even under oracle routing, 632 of 1,535 species retain F1$=$0 (compared to 804 under the flat baseline). The majority of these species have insufficient per-species training examples to acquire discriminative representations at the current scale; the full 50M-read evaluation is expected to reduce, but not eliminate, this long-tail problem.

4.12.5 Per-Genus Species Classifiers (100K-read Subset)

As a complementary architecture to the single flat sp_v4 classifier, we trained 81 dedicated per-genus species classifiers using a frozen NT-v2 backbone and a lightweight attention-pooling head (Section 3.6.3). Each classifier is responsible only for discriminating among the species within its assigned genus (average 12.8 species per genus); 39 single-species genera require no classifier and always return the unique species deterministically. Training used a 5M-read balanced subset (approximately 3,260 reads per species) with backbone weights frozen throughout.

The complete 100K-read evaluation compares all six modes: four routing modes applied to the single sp_v4 model (Table 4.23) and two per-genus pipeline modes. All experiments use the same v9 genus model (epoch 30, 65.7% subset accuracy) for genus assignment. Table 4.24 summarises the results.

Table 4.24: Complete hierarchical species evaluation on 100K-read subset. Flat, Hard Top-5, Soft, and Oracle modes use sp_v4 (ep26); Per-genus modes use dedicated per-genus NT-v2 classifiers trained on the 5M balanced subset.

Model	Mode	Top-1	Top-5	Top-10	F1 (macro)
sp_v4 (ep26)	Flat (baseline)	16.4%	36.7%	46.7%	0.137
	Hard Top-5 routing	16.5%	36.4%	46.1%	0.139
	Soft routing	16.2%	36.6%	46.2%	0.135
	Oracle (true genus)	27.9%	55.4%	65.3%	0.256
Per-genus (5M)	Predicted genus (v9)	11.2%	26.0%	32.9%	0.109
	Oracle genus (true)	21.8%	48.7%	60.2%	0.204

Genus routing errors dominate the per-genus pipeline. With predicted genus routing (v9, 65.7% accuracy), the per-genus pipeline achieves only 11.2% Top-1—5.2 pp *below* the flat sp_v4 baseline. The 34.3% genus error rate is more damaging here than in the routing modes applied to sp_v4 logits: when the wrong genus model is selected, every

read in that genus is classified against the wrong species set, producing near-zero accuracy for those reads. By contrast, sp_v4 Hard Top-5 routing retains some partial signal from the flat logits even when the top-1 genus is wrong.

Per-genus oracle underperforms sp_v4 oracle. Even under perfect genus assignment, per-genus oracle achieves 21.8%—6.1 pp below sp_v4 oracle (27.9%). This gap reflects the weaker individual per-genus classifiers: each was trained on ∼3,260 reads per species (compared to ∼32,600 for sp_v4) with a frozen backbone rather than LoRA fine-tuning. The per-genus approach would likely exceed the sp_v4 oracle if each classifier received 50M-scale training with an adapted backbone.

Long-tail coverage improves substantially. Despite the lower Top-1 accuracy, the per-genus architecture reduces the number of species with F1=0 from 804 (flat sp_v4) to 401 under predicted routing and 362 under oracle routing. This suggests that the per-genus framing—where each rare species competes only within its genus rather than against all 1,535 classes—provides a more favourable learning signal for rare species, even at reduced training scale.

4.13 Summary of Findings

The experimental results lead to several key conclusions:

1. **Data volume dominates**: Training data volume is the single most important factor for metagenomic classification accuracy. A $10\times$ data increase yields $+7.76$ pp, while training technique optimizations yield at most ± 0.5 pp.

2. **RC TTA is universally beneficial**: Reverse-complement test-time augmentation

consistently improves accuracy by $+1$–1.5 pp across all settings, exploiting the biological strand symmetry of DNA.

3. **Class balance matters**: Balanced sampling across species eliminates the "dead class" problem (F1=0 classes: $9 \rightarrow 2$) and dramatically improves macro F1 ($+12.27$ pp).

4. **Overfitting signals data starvation**: The transition from 6.25% to 1.30% train-validation gap when scaling from 500K to 5M reads confirms that the model was severely data-limited at the smaller scale.

5. **Pre-trained backbone provides a substantial advantage**: The controlled ablation (v9 vs. v11) shows a 12.18 pp accuracy gap in favor of the 29-layer pre-trained NT-v2 backbone over a single-layer randomly initialized Transformer at the same 50M data scale, confirming that pre-training on 850 reference genomes provides representations that meaningfully improve read discrimination.

6. **Hardware is sufficient**: Current NVIDIA H100 hardware is adequately provisioned; the operational bottleneck is the SLURM time limit rather than computational resources.

7. **Per-genus classifiers reduce long-tail failures**: Dedicated per-genus species classifiers reduce F1=0 species from 804 to 362–401, confirming that genus-scoped classification provides a more favourable learning signal for rare taxa. However, overall Top-1 accuracy requires either higher genus accuracy ($>$80%) or stronger per-genus models (50M-scale training with LoRA) to surpass the flat sp_v4 baseline.

Chapter 5

Conclusion and Future Work

5.1 Summary of Contributions

This thesis has investigated the application of genomic foundation models to metagenomic taxonomic classification through a token-level representation approach. Our main findings are:

Token-level representations improve classification. By preserving the full sequence of token embeddings from the Nucleotide Transformer v2 backbone—rather than discarding positional information through mean pooling—and aggregating them via a learnable attention-pooling head, we achieve genus-level accuracy improvements over frozen-embedding baselines. The token-level approach with LoRA fine-tuning achieves 63.05% RC TTA accuracy on 120 genera with 5M balanced training reads, scaling to 67.07% with 50M reads, compared to 51.70% for the frozen mean-pool baseline.

Data volume is the dominant performance factor within a fixed architecture. Through systematic data scaling experiments spanning 500K to 50M training reads, we demonstrate that increasing training data volume by $10\times$ yields $+7.76$ percentage points in accuracy and $+12.27$ percentage points in macro F1, while extensive architectural and training technique ablations yield at most ± 0.5 percentage points. This finding aligns with the observations of Wichmann et al. [8], who achieve 98.3% genus recall (98.9% precision) using a large-scale balanced training dataset.

Tokenization is the dominant cross-architecture design factor. Controlled ab-

lations training MetaTransformer [8] on the same 50M balanced dataset under six tokenization configurations ($k \in \{6, 12, 13\}$, stride $\in \{1, k\}$) reveal two compounding effects: overlapping stride=1 is critical (up to $+60$ pp over non-overlapping at $k = 13$), and longer k yields monotonically higher accuracy among overlapping configurations. MetaTransformer with overlapping 13-mer tokenization, trained from scratch, achieves **87.42%** genus Top-1—**+20.35 pp** above NT-v2 + LoRA with RC TTA (67.07%)—and **49.62%** species Top-1 (1,535 classes). This result demonstrates that task-specific tokenization design is a more decisive factor than pre-trained prior knowledge. The primary limitation of our GFM-based approach is therefore NT-v2's 6-mer tokenization constraint, not its backbone capacity or adaptation strategy.

Class balance is critical for rare taxa. Balanced subsampling across species reduces the number of F1=0 classes from 9 to 2 and improves macro F1 by $+12.27$ percentage points, demonstrating that class imbalance—not model capacity—is the primary cause of poor performance on rare taxa.

RC TTA is a robust, free improvement. Reverse-complement test-time augmentation consistently improves accuracy by $+1$ to $+1.5$ percentage points across all experimental conditions, requiring no additional training cost.

Current hardware is sufficient. The NVIDIA H100 80 GB GPU is adequately provisioned for this workload, with VRAM utilization below 50% even at batch size 256. The operational bottleneck is the SLURM time limit rather than computational resources.

5.2 Limitations

This work has several limitations that should be acknowledged:

1. **Simulated data only**: All experiments use reads simulated by ART from reference genomes. Real metagenomic reads contain additional noise (host contamination, chimeric reads, strain-level variation) that may degrade classifier performance.

2. **Limited taxonomic scope**: The reference database covers 120 genera and 1,535 species of human gut microorganisms. Performance on environmental samples with broader taxonomic diversity is unknown.

3. **Single read length**: All experiments use 150 bp reads. Performance at other read lengths (e.g., 250 bp for Illumina MiSeq, or long reads from Oxford Nanopore / PacBio) has not been evaluated.

4. **Incomplete data scaling**: Training on the full 258M dataset was not completed at the time of writing, leaving the upper bound of the scaling trajectory unconfirmed. The v9 (50M) experiment is complete, achieving 67.07% RC TTA accuracy.

5. **No streaming data loader**: Current implementation loads all training reads into memory, limiting scalability beyond $\sim$50M reads without substantial RAM ($>$50 GB).

6. **Architectural capacity ceiling**: MetaTransformer's 5M-parameter, single-encoder design saturates within one training epoch on 50M reads; a second epoch produces overfitting rather than further generalisation. A larger model would likely improve beyond 78.83% but was not evaluated due to infrastructure constraints.

5.3 Future Work

Several promising directions emerge from this work:

5.3.1 Domain-Specific Pre-trained Foundation Models

The tokenization ablation (Section 4.11) establishes that overlapping 12-mer tokenization is more beneficial than 6-mer pre-training for this task. The ideal direction is therefore a foundation model pre-trained with overlapping 12-mer (or similarly long) tokenization on large microbial genome collections—no such publicly available model currently exists. Promising avenues include:

- Pre-training a transformer on the GTDB database [32] (>400,000 microbial genomes) with 12-mer tokenization, which would combine the discriminative power of long k-mers with the representational quality of large-scale pre-training.
- Fine-tuning NT-v2 with a learned 12-mer projection layer to bridge the tokenization gap without full re-pre-training, though this approach may be limited by the model's pre-trained 6-mer positional encodings.

5.3.2 Full-Scale Data Training

The most immediate next step is completing training on the full 258M-read dataset. Our data scaling analysis predicts that this could yield genus-level accuracy in the 82–90% range, substantially narrowing the gap with MetaTransformer. Achieving this requires either:

- Implementing a streaming data loader that reads from disk, eliminating the in-memory requirement.
- Requesting extended SLURM time limits (72+ hours) or developing a multi-job training orchestration system.

5.3.3 Species-Level Scaling and Hierarchical Classification

Species classification at the 500K scale achieved only ∼8% accuracy (1,535 classes), with oracle-genus evaluation capping at ∼21%. Scaling to 50M balanced reads (sp_v4, Section 4.12.2) lifted forward-only Top-1 accuracy to 17.55%, with Top-5 reaching 38.19% and only 191 of 1,535 species retaining F1=0. The hierarchical classification pipeline —genus prediction via the v9 model, followed by species classification within the candidate set—was validated on a 100K held-out subset (Section 4.12.4): oracle-genus routing raised Top-1 to 27.9% (+11.5 pp), while top-k and soft routing yielded negligible gains under the current 65.7%–67% genus accuracy regime. Meaningful real routing improvement therefore requires pushing genus accuracy substantially higher, which in turn motivates the tokenization study below. Full 5M-scale evaluation of the routing modes was constrained by the 200 GB per-job memory ceiling on TWCC and is deferred to a streaming-metrics implementation of the evaluation pipeline.

5.3.4 Tokenization Study

The controlled ablation (Section 4.11) establishes overlapping large-k tokenization as the dominant design factor. MetaTransformer 13-mer stride=1 achieves 87.42% genus and 49.62% species accuracy trained from scratch, surpassing NT-v2 + LoRA by 20.35 pp. Remaining open questions include:

- Does RC TTA provide an equivalent benefit ($\sim +1$ pp) for 12-mer or 13-mer models, and what is the combined accuracy ceiling with TTA?
- Is $k = 13$ the optimum, or do larger k values (e.g. $k = 15$ or $k = 21$) continue to improve? The monotonic trend across $k \in \{6, 12, 13\}$ suggests further gains may

be possible.

- Would BPE tokenization—which learns variable-length tokens from data—discover taxonomically informative motifs of varying length more efficiently than fixed k-mer windows?

- A 13-mer pre-trained genomic foundation model would combine the tokenization advantage with representational pre-training quality. Building or fine-tuning such a model is the most direct path to exceeding the current 87.42% ceiling.

5.3.5 Architectural Improvements

Multi-layer feature fusion: Current experiments use only the last hidden layer of NT-v2. Combining representations from multiple layers (e.g., via weighted averaging or learned gating) could capture both low-level sequence features and high-level semantic information [9].

Adapter fusion: Training separate LoRA adapters for genus and species tasks and combining them through adapter fusion [33] could enable joint hierarchical classification without duplicating the backbone.

5.3.6 Real-World Validation

Validating the classifier on real metagenomic sequencing data from public repositories (e.g., Human Microbiome Project [34]) is essential for assessing practical applicability. This includes evaluating robustness to:

- Sequencing errors and quality variations across platforms.

- Host DNA contamination.

- Novel or uncharacterized taxa absent from the training database (out-of-distribution detection).

5.3.7 Efficiency Optimization

For practical deployment in clinical or environmental monitoring settings, inference speed is critical. Potential optimizations include:

- **Model distillation**: Distilling the 498M-parameter LoRA-adapted model into a smaller task-specific model (e.g., 50M parameters).

- **Quantization**: Applying INT8 or INT4 quantization to reduce memory and accelerate inference.

- **Batched inference**: Optimizing the inference pipeline for high-throughput batch processing of millions of reads.

5.3.8 Pre-training Strategy

The current work fine-tunes a model pre-trained on 850 genomes from 174 species. A foundation model pre-trained specifically on a larger, more diverse collection of microbial genomes (e.g., the full GTDB database with >400,000 genomes [32]) may provide more suitable representations for metagenomic classification, reducing the amount of task-specific fine-tuning data required.

5.4 Concluding Remarks

This thesis demonstrates both the promise and the current limitations of applying genomic foundation models to metagenomic taxonomic classification. The controlled ablation experiments yield two concrete findings that extend beyond the specific system studied here.

First, pre-training on large genomic corpora provides a substantial and measurable advantage over training from scratch when tokenization is held constant: NT-v2 + LoRA outperforms a MetaTransformer of comparable trainable parameter count by +18.29 pp at 50M reads under identical 6-mer tokenization. This validates the GFM pre-training paradigm for this task.

Second, and more significantly, tokenization design dominates architectural choice. Among overlapping stride=1 configurations, genus Top-1 accuracy improves monotonically with k-mer length: 6-mer (48.87%) $<$ 12-mer (78.83%) $<$ 13-mer (87.42%). MetaTransformer with overlapping 13-mer tokenization, trained *from scratch* on the same 50M reads, achieves 87.42% genus Top-1 and 49.62% species Top-1 (1,535 classes)—surpassing NT-v2 + LoRA with RC TTA (67.07%) by **+20.35 pp** despite having no pre-training. The principal limitation of NT-v2-based approaches is therefore the 6-mer tokenization constraint inherited from pre-training, rather than the backbone's capacity or the LoRA adaptation strategy.

Together, these findings point to a clear direction: a foundation model pre-trained with overlapping 13-mer (or longer) tokenization on diverse microbial genomes would combine the tokenization advantage demonstrated here with the representational quality of large-scale pre-training. Realising this direction, along with completing the species-level

scaling experiments and hierarchical classification pipeline, represents the most promising path toward a competitive, generalisable GFM-based metagenomic classifier.

References

[1] C. Quince, A. W. Walker, J. T. Simpson, N. J. Loman, and N. Segata, "Shotgun metagenomics, from sampling to analysis," *Nature Biotechnology*, vol. 35, no. 9, pp. 833–844, 2017. DOI: 10.1038/nbt.3935

[2] S. H. Ye, K. J. Siddle, D. J. Park, and P. C. Sabeti, "Benchmarking metagenomics tools for taxonomic classification," *Cell*, vol. 178, no. 4, pp. 779–794, 2019. DOI: 10.1016/j.cell.2019.07.010

[3] F. P. Breitwieser, J. Lu, and S. L. Salzberg, "A review of methods and databases for metagenomic classification and assembly," *Briefings in Bioinformatics*, vol. 20, no. 4, pp. 1125–1136, 2019. DOI: 10.1093/bib/bbx120

[4] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990. DOI: 10.1016/S0022-2836(05)80360-2

[5] B. Buchfink, C. Xie, and D. H. Huson, "Fast and sensitive protein alignment using DIAMOND," *Nature Methods*, vol. 12, no. 1, pp. 59–60, 2015. DOI: 10.1038/nmeth.3176

[6] D. E. Wood, J. Lu, and B. Langmead, "Improved metagenomic analysis with Kraken 2," *Genome Biology*, vol. 20, p. 257, 2019. DOI: 10.1186/s13059-019-1891-0

[7] D. Kim, L. Song, F. P. Breitwieser, and S. L. Salzberg, "Centrifuge: Rapid and sensitive classification of metagenomic sequences," *Genome Research*, vol. 26, no. 12, pp. 1721–1729, 2016. DOI: 10.1101/gr.210641.116

[8] A. Wichmann, E. Kanzler, and G. Zeller, "MetaTransformer: Deep metagenomic sequencing read classification with self-attention based neural network," *bioRxiv*, 2023. DOI: 10.1101/2023.12.28.573langchain

[9] H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla, N. L. Carranza, A. H. Grzywaczewski, F. Ober, C. Dallago, E. Hassan, A. Grber, M. Trop, et al., "The Nucleotide Transformer: Building and evaluating robust foundation models for human genomics,"

Nature Methods, 2024, Preprint on bioRxiv, 2023. DOI: 10.1038/s41592-024-02523-z

[10] Y. Ji, Z. Zhou, H. Liu, and R. V. Davuluri, "DNABERT: Pre-trained bidirectional encoder representations from transformers model for DNA-language in genome," *Bioinformatics*, vol. 37, no. 15, pp. 2112–2120, 2021. DOI: 10.1093/bioinformatics/btab083

[11] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu, "DNABERT-2: Efficient foundation model and benchmark for multi-species genome," *arXiv preprint arXiv:2306.15006*, 2024.

[12] E. Nguyen, M. Poli, M. G. Durrant, A. W. Thomas, B. Kang, J. Sullivan, M. Y. Ng, A. Lewis, A. Patel, A. Lou, et al., "Sequence modeling and design from molecular to genome scale with Evo," *Science*, vol. 386, no. 6723, 2024. DOI: 10.1126/science.ado9336

[13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[14] Z. Zhang, S. Schwartz, L. Wagner, and W. Miller, "A greedy algorithm for aligning DNA sequences," *Journal of Computational Biology*, vol. 7, no. 1–2, pp. 203–214, 2000. DOI: 10.1089/10665270050081478

[15] Q. Liang, P. W. Bible, Y. Liu, B. Zou, and L. Wei, "DeepMicrobes: Taxonomic classification for metagenomics with deep learning," *NAR Genomics and Bioinformatics*, vol. 2, no. 1, 2020. DOI: 10.1093/nargab/lqaa009

[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[17] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[18] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smestad, A. Costa, M. Fazel-Zarandi, T. Sercu, S. Candela, and A. Rives, "Evolutionary-scale prediction of atomic-level protein structure with a language model," *Science*, vol. 379, no. 6637, pp. 1123–1130, 2023. DOI: 10.1126/science.ade2574

[19] M. Poli, J. Wang, S. Massaroli, J. Quesnelle, R. Carlow, E. Nguyen, and C. Re, "Hyena Hierarchy: Towards larger convolutional language models," *arXiv preprint arXiv:2302.10866*, 2023.

[20] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning (ICML)*, 2019, pp. 2790–2799.

[21] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *arXiv preprint arXiv:2101.00190*, 2021.

[22] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu, "Towards a better understanding of reverse-complement equivariance for deep learning models in genomics," in *Proceedings of the Machine Learning in Computational Biology (MLCB) Workshop, PMLR*, vol. 165, 2022. [Online]. Available: https://proceedings.mlr.press/v165/zhou22a.html

[23] M. Ma, "Reverse-complement consistency for DNA language models," *arXiv preprint arXiv:2509.18529*, 2025.

[24] A. Brown and T. Bepler, "Reverse-complement equivariant networks for DNA sequences," *bioRxiv*, 2021. DOI: 10.1101/2021.06.03.446953

[25] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.

[26] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *International Conference on Learning Representations (ICLR)*, 2021.

[27] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127 063, 2024. DOI: 10.1016/j.neucom.2023.127063

[28] A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira, "Perceiver: General perception with iterative attention," in *International Conference on Machine Learning (ICML)*, 2021, pp. 4651–4664.

[29] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant input," in *International Conference on Machine Learning (ICML)*, 2019, pp. 3744–3753.

[30] W. Huang, L. Li, J. R. Myers, and G. T. Marth, "ART: A next-generation sequencing read simulator," *Bioinformatics*, vol. 28, no. 4, pp. 593–594, 2012. DOI: 10.1093/bioinformatics/btr708

[31] A. Almeida, S. Nayfach, M. Boland, F. Strozzi, M. Beracochea, Z. J. Shi, K. S. Pollard, E. Sakharova, D. H. Parks, P. Hugenholtz, et al., "A unified catalog of 204,938 reference genomes from the human gut microbiome," *Nature Biotechnology*, vol. 39, no. 1, pp. 105–114, 2021. DOI: 10.1038/s41587-020-0603-3

[32] D. H. Parks, M. Chuvochina, C. Rinke, A. J. Mussig, P.-A. Chaumeil, and P. Hugenholtz, "GTDB: An ongoing census of bacterial and archaeal diversity through a phylogenetically consistent, rank normalized and complete genome-based taxonomy," *Nucleic Acids Research*, vol. 50, no. D1, pp. D199–D207, 2022. DOI: 10.1093/nar/gkab776

[33] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," *arXiv preprint arXiv:2005.00247*, 2021.

[34] Human Microbiome Project Consortium, "Structure, function and diversity of the healthy human microbiome," *Nature*, vol. 486, no. 7402, pp. 207–214, 2012. DOI: 10.1038/nature11234

Appendix A

Experiment Configuration Details

This appendix provides the full configuration files used for the key experiments described in Chapter 4.

A.1 v8 Genus Configuration (5M Balanced)

```
model:
  backbone: InstaDeepAI/nucleotide-transformer-v2-500m-multi-
     species
  head_type: attention_pool
  num_query_tokens: 4
  hidden_dim: 512
  dropout: 0.15

lora:
  r: 16
  alpha: 32
  dropout: 0.05
  target_modules: [query, key, value]

data:
  fasta_path: /path/to/balanced_5M/reads.fa
  labels_path: /path/to/balanced_5M/labels.tsv
  classification_level: genus
  max_length: 32
  val_ratio: 0.1

training:
  batch_size: 32
```

```
  gradient_accumulation_steps: 2
  num_epochs: 20
  phase1_epochs: 5
  backbone_lr: 3.0e-5
  head_lr: 5.0e-4
  weight_decay: 0.02
  warmup_ratio: 0.1
  early_stopping_patience: 7
  rc_augmentation: true
```

Listing A.1: YAML configuration for v8 genus experiment.

A.2 v9 Genus Configuration (50M Balanced)

```
model:
  backbone: InstaDeepAI/nucleotide-transformer-v2-500m-multi-
     species
  head_type: attention_pool
  num_query_tokens: 4
  hidden_dim: 512
  dropout: 0.15

lora:
  r: 16
  alpha: 32
  dropout: 0.05
  target_modules: [query, key, value]

data:
  fasta_path: /path/to/balanced_50M/reads.fa
  labels_path: /path/to/balanced_50M/labels.tsv
  classification_level: genus
  max_length: 32
```

```yaml
  val_ratio: 0.1

training:
  batch_size: 256
  gradient_accumulation_steps: 1
  num_epochs: 10
  phase1_epochs: 0
  backbone_lr: 3.0e-5
  head_lr: 5.0e-4
  weight_decay: 0.02
  warmup_ratio: 0.1
  early_stopping_patience: 7
  rc_augmentation: true
```

Listing A.2: YAML configuration for v9 genus experiment.

A.3 SLURM Job Script Example

```bash
#!/bin/bash
#SBATCH --job-name=gfm-genus-v9
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=4
#SBATCH --mem=128G
#SBATCH --time=48:00:00
#SBATCH --output=slurm-%j.out

module load cuda/12.1
source activate gfm

python scripts/train.py \
  --config configs/nt_token_genus_v9_50M.yaml \
  --init_from results/nt_token_genus_lora_v8_5M/best.pt
```

```
16
17 python scripts/evaluate.py \
18   --config configs/nt_token_genus_v9_50M.yaml \
19   --checkpoint results/nt_token_genus_lora_v9_50M/best.pt
20
21 python scripts/evaluate.py \
22   --config configs/nt_token_genus_v9_50M.yaml \
23   --checkpoint results/nt_token_genus_lora_v9_50M/best.pt \
24   --rc_tta
```

Listing A.3: SLURM script for training on TWCC cluster.